

Table of Contents

1 DIARIO DE TRABAJO FIN DE GRADO.

1.1 RealStateAI - Herramienta de Análisis de Inversión Inmobiliaria

Autor: Alejandro Zabaleta **Fecha de Inicio:** Curso 2025-2026 **Última Actualización:** 12 de Febrero de 2026 **Repositorio GitHub:** <https://github.com/ZabaHD4K/CalculadoraRentabilidadInmobiliaria> **Universidad:** U-tad

1.2 ÍNDICE

1. Resumen Ejecutivo
 2. Estado de la Cuestión
 3. Justificación y Motivación
 4. Objetivos del Proyecto
 5. Análisis de Requisitos
 6. Casos de Uso
 7. Metodología
 8. Planificación y Cronograma
 9. Arquitectura del Sistema
 10. Desarrollo e Implementación
 11. Validación y Pruebas
 12. Riesgos y Consideraciones Éticas
 13. Estado Actual del Proyecto
 14. Conclusiones Preliminares
 15. Bibliografía
-

1.3 RESUMEN EJECUTIVO

RealStateAI es una aplicación web de análisis de inversión inmobiliaria que combina automatización, inteligencia artificial generativa y cálculos financieros para ayudar a inversores a tomar decisiones informadas sobre propiedades de alquiler en el mercado español.

1.3.1 Problema Identificado

Los inversores inmobiliarios individuales en España carecen de herramientas integradas y accesibles para analizar la rentabilidad real de sus inversiones. Según el informe del Instituto de Estudios Económicos (IEE, 2024), la complejidad fiscal del mercado inmobiliario español — con 17 comunidades autónomas aplicando tipos impositivos diferentes para el ITP— supone

una barrera significativa para el inversor particular. Las alternativas actuales presentan limitaciones claras:

- **Hojas de cálculo manuales:** propensas a errores de fórmula y requieren conocimiento fiscal especializado.
- **Portales inmobiliarios (Idealista, Fotocasa):** ofrecen listados pero no calculan rentabilidad real ni integran gastos fiscales.
- **Herramientas profesionales (PlanRadar, PropTech):** orientadas a grandes empresas con costes de licencia elevados (desde 49€/mes en adelante), inadecuadas para el inversor particular.

1.3.2 Solución Propuesta

Una plataforma web que: 1. Extrae datos de propiedades desde portales inmobiliarios mediante técnicas de web scraping asistido por IA. 2. Calcula todos los gastos asociados a la compra: impuestos (ITP/IVA/AJD), notaría, registro y gestoría. 3. Utiliza modelos de lenguaje (LLM) para estimaciones de alquiler y análisis de características. 4. Calcula métricas financieras avanzadas: ROI, TIR, VAN, cash flow y tabla de amortización. 5. Proporciona un dashboard financiero interactivo con gráficos y simulaciones.

1.4 ESTADO DE LA CUESTIÓN

1.4.1 1. El Sector PropTech en España

El sector PropTech (Property Technology) ha experimentado un crecimiento significativo en España durante la última década. Según el mapa PropTech elaborado por la consultora Finnovating (2024), existen más de 500 startups PropTech en España, pero la mayoría se concentran en la intermediación y gestión de activos para grandes carteras, dejando desatendido al inversor particular.

Asensio-Soto (2023) señala en su tesis doctoral sobre la digitalización de la intermediación inmobiliaria en España que el sector PropTech ha avanzado principalmente en la fase de comercialización (portales de anuncios y plataformas de intermediación online), pero existe un déficit notable en herramientas de análisis financiero accesibles para el pequeño inversor.

1.4.2 2. Análisis de Herramientas Existentes

1.4.2.1 Herramientas Tradicionales: Hojas de Cálculo

El uso de hojas de cálculo como herramienta principal de análisis financiero inmobiliario sigue siendo predominante entre inversores particulares. Un estudio de la plataforma de formación Libertad Inmobiliaria revela que una gran parte de los asistentes a sus cursos depende de plantillas Excel personalizadas para sus cálculos (Libertad Inmobiliaria, 2024).

- **Ventajas:** flexibilidad, personalización, sin coste adicional.
- **Desventajas:** propensas a errores de fórmula, no automatizadas, requieren conocimiento fiscal por comunidad autónoma, difíciles de mantener actualizadas.

1.4.2.2 Portales Inmobiliarios (Idealista, Fotocasa)

Los principales portales inmobiliarios españoles ofrecen funcionalidades de búsqueda y listado de propiedades, pero carecen de herramientas de análisis de inversión integradas (Idealista, 2025):

- Información de precios de venta y alquiler por separado.
- No calculan rentabilidad real neta.
- No integran gastos fiscales por comunidad autónoma.
- No ofrecen simulación de hipoteca contextualizada.

1.4.2.3 Herramientas Profesionales

Existen soluciones profesionales como PlanRadar, CBRE Analytics o JLL Research que ofrecen análisis completos, pero están orientadas a fondos de inversión y grandes promotoras, con costes de licencia que las hacen inaccesibles para el inversor individual (Asensio-Soto, 2023).

1.4.3 3. Uso de LLMs en el Sector Inmobiliario

La aplicación de modelos de lenguaje de gran escala (LLMs) al sector inmobiliario es un área de investigación emergente. Estudios recientes han explorado el uso de GPT-4 y modelos similares para la estimación de precios inmobiliarios con resultados prometedores pero con limitaciones significativas.

Geerts et al. (2025) demuestran que los LLMs pueden aprovechar variables hedónicas de las propiedades (superficie, ubicación, equipamientos) para producir estimaciones de precio significativas, aunque los modelos de machine learning tradicionales superan a los LLMs en precisión predictiva pura. Los autores señalan además que los LLMs presentan sobreconfianza en los intervalos de predicción y capacidades limitadas de razonamiento espacial, pero destacan su potencial para mejorar la transparencia en la valoración inmobiliaria.

Es importante señalar las **limitaciones inherentes** de este enfoque: - Los LLMs no tienen acceso a datos de transacciones reales en tiempo real. - Las estimaciones dependen de la calidad y actualización de los datos de entrenamiento. - Los resultados deben considerarse orientativos y complementarse con datos de mercado reales.

1.4.4 4. Conclusión del Estado de la Cuestión

Existe un **hueco de mercado** identificado: no hay una herramienta accesible, automatizada e integral para el inversor inmobiliario particular en España que combine: - Extracción automatizada de datos de propiedades. - Cálculos fiscales precisos por comunidad autónoma. - Análisis de rentabilidad integral con métricas financieras avanzadas. - Interfaz intuitiva con simulaciones interactivas.

1.5 JUSTIFICACIÓN Y MOTIVACIÓN

1.5.1 Motivación Personal

El interés por la inversión inmobiliaria surge de la observación directa del entorno: en conversaciones con familiares y conocidos que han invertido en vivienda para alquilar, se detectó un patrón recurrente: la mayoría calculaban la rentabilidad de forma simplificada (dividiendo el alquiler anual entre el precio de compra) sin tener en cuenta los gastos reales asociados —impuestos de transmisión, notaría, registro, IBI, comunidad, seguros, periodos vacantes— lo que llevaba a sobreestimar la rentabilidad real de sus inversiones. Esta brecha entre la rentabilidad percibida y la rentabilidad real motivó la idea de crear una herramienta que automatizase estos cálculos de forma completa y accesible.

Además, como estudiante de ingeniería de software, este proyecto representaba una oportunidad de integrar múltiples áreas de conocimiento: desarrollo web full-stack, integración de APIs de inteligencia artificial, cálculos financieros, fiscalidad española y diseño de interfaces interactivas, todo en un producto con utilidad real.

El mercado inmobiliario es el vehículo de inversión más popular en España según el Banco de España (2024), pero el análisis de rentabilidad real es complejo y requiere considerar múltiples variables que a menudo son pasadas por alto por inversores particulares: fiscalidad autonómica, gastos notariales, seguros, periodos vacantes y mantenimiento.

1.5.2 Necesidad Detectada

A través de la investigación realizada en el estado de la cuestión, se identificó que:

1. Los inversores particulares **subestiman sistemáticamente los gastos** asociados a la compra (ITP, notaría, registro, gestoría) según el análisis de la OCU sobre gastos de compraventa de vivienda (OCU, 2023).
2. **No existe una herramienta gratuita y completa** que integre cálculos fiscales autonómicos con análisis de rentabilidad.
3. La fragmentación de información entre portales, calculadoras de hipotecas y normativa fiscal dificulta la toma de decisiones informada.
4. Las herramientas profesionales existentes están fuera del alcance económico del inversor particular.

1.5.3 Aportación del TFG

Este proyecto aporta: 1. **Automatización inteligente** mediante integración de LLMs con web search. 2. **Cálculos fiscales precisos** con tipos ITP actualizados para las 17 comunidades autónomas, ciudades autónomas de Ceuta y Melilla, y diferenciación entre obra nueva (IVA+AJD) y segunda mano (ITP). 3. **Métricas financieras avanzadas** (ROI, TIR, VAN, cash flow, payback period, tabla de amortización). 4. **Dashboard financiero interactivo** con gráficos y simulaciones de escenarios. 5. **Código abierto** que puede beneficiar a la comunidad.

1.6 OBJETIVOS DEL PROYECTO

1.6.1 Objetivo General

Desarrollar una aplicación web que permita a inversores inmobiliarios particulares analizar la rentabilidad real de propiedades de alquiler en España de forma automatizada, precisa y completa, integrando extracción de datos, cálculos fiscales autonómicos, estimaciones asistidas por IA y métricas financieras avanzadas.

1.6.2 Objetivos Específicos

1.6.2.1 1. Objetivos Técnicos

- ☒ Implementar arquitectura cliente-servidor con Node.js/Express (backend) y Next.js 16/React 19 (frontend).
- ☒ Integrar OpenAI API (GPT-5-mini con web search y GPT-4o) para extracción de datos y estimaciones.
- ☒ Desarrollar sistema de extracción de datos de propiedades desde URLs de Idealista.
- ☒ Implementar API REST completa con CRUD de propiedades.
- ☒ Implementar sistema de autenticación básico con contraseña hasheada (SHA-256).
- ☒ Desplegar frontend en Vercel y backend en Railway.
- ☒ Integrar consulta del Euribor en tiempo real desde la API del Banco de España.

1.6.2.2 2. Objetivos Funcionales

- ☒ Extracción automática de datos desde URLs de Idealista (precio, superficie, habitaciones, dirección, descripción, características).
- ☒ Cálculo de impuestos (ITP/IVA/AJD) por comunidad autónoma con tipos actualizados.
- ☒ Cálculo de gastos de compra (notaría, registro, gestoría, tasación).
- ☒ Estimación de alquiler mensual mediante IA (GPT-5-mini con web search; GPT-4o como fallback).
- ☒ Estimación de gastos anuales de vivienda mediante IA (IBI, comunidad, seguros).
- ☒ Cálculo de hipoteca con sistema francés de amortización (fija y variable con Euribor).
- ☒ Dashboard financiero con: ROI, TIR, VAN, cash flow, rentabilidad bruta/neta, payback period.
- ☒ Gráficos interactivos: evolución de rentabilidad a 10 años, desglose de gastos (pie chart), tabla de amortización.
- ☒ Gestión de múltiples propiedades con CRUD completo.
- ☐ Persistencia con base de datos (actualmente en memoria).
- ☐ Exportación de informes PDF.

1.6.2.3 3. Objetivos de Usabilidad

- ☒ Interfaz intuitiva con diseño dark mode (Slate + Teal).
- ☒ Diseño responsive.
- ☒ Feedback visual en tiempo real (loading states, animaciones).
- ☒ Validación de datos en formularios.

- ☒ Simulación interactiva de escenarios financieros (sliders para editar gastos, hipoteca, inflación).

1.7 ANÁLISIS DE REQUISITOS

1.7.1 Requisitos Funcionales

ID	Requisito	Prioridad	Estado
RF1	Añadir propiedades mediante URL de Idealista	Alta	Completado
RF2	Extraer automáticamente datos de la propiedad	Alta	Completado
RF3	Calcular impuestos según comunidad autónoma	Alta	Completado
RF4	Calcular gastos notariales y de registro	Alta	Completado
RF5	Estimar precio de alquiler mediante IA	Media	Completado
RF6	Calcular cuota de hipoteca (fija y variable)	Alta	Completado
RF7	Calcular gastos anuales de vivienda	Alta	Completado
RF8	Editar datos de propiedades	Media	Completado
RF9	Almacenar múltiples propiedades	Alta	Completado
RF10	Eliminar propiedades	Media	Completado
RF11	Dashboard con todas las propiedades y ROI	Alta	Completado
RF12	Consultar Euribor actual automáticamente	Media	Completado
RF13	Dashboard financiero con métricas avanzadas	Alta	Completado
RF14	Gráficos interactivos de rentabilidad	Alta	Completado
RF15	Simulación de escenarios financieros	Alta	Completado
RF16	Sistema de autenticación	Media	Completado
RF17	Persistencia en base de datos	Media	Planificado para fase posterior

1.7.2 Requisitos No Funcionales

ID	Categoría	Requisito
RNF1	Rendimiento	Análisis de URL completado en menos de 60 segundos
RNF2	Rendimiento	Dashboard financiero carga en menos de 2 segundos
RNF3	Usabilidad	Interfaz intuitiva sin necesidad de manual
RNF4	Usabilidad	Diseño responsive funcional en móviles y tablets
RNF5	Seguridad	API keys almacenadas en variables de entorno
RNF6	Seguridad	Contraseña de acceso hasheada (SHA-256)
RNF7	Seguridad	CORS configurado con validación dinámica de orígenes
RNF8	Mantenibilidad	Código modular con separación frontend/backend
RNF9	Mantenibilidad	TypeScript para type-safety en el frontend
RNF10	Disponibilidad	Despliegue en Vercel (frontend) y Railway (backend)

1.8 CASOS DE USO

El siguiente diagrama muestra las relaciones entre el actor principal (Usuario Inversor) y los nueve casos de uso identificados para el sistema, incluyendo las relaciones «include» y «extend» entre ellos.

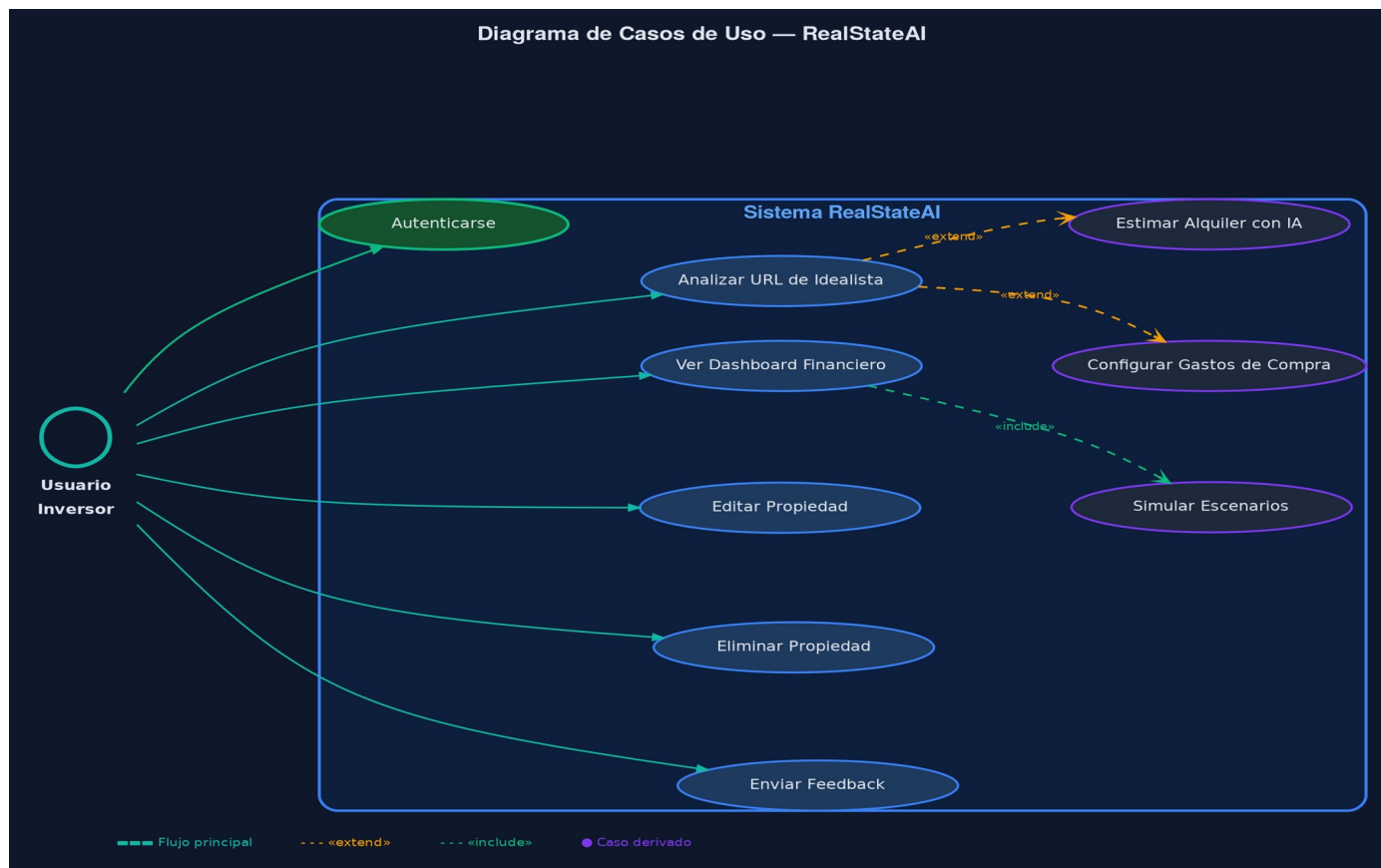


Figura: Diagrama de Casos de Uso — RealStateAI

1.8.1 CU-001: Analizar Propiedad desde URL

Actor Principal: Usuario Inversor

Precondiciones: - El usuario está autenticado. - El usuario tiene una URL válida de Idealista.

Flujo Principal: 1. El usuario hace clic en “Añadir Propiedad”. 2. El sistema muestra el formulario de nueva propiedad. 3. El usuario pega la URL de Idealista. 4. El usuario hace clic en “Analizar URL”. 5. El sistema envía la URL al backend. 6. El backend utiliza GPT-5-mini con web search para acceder a la URL y extraer datos. 7. El backend parsea la respuesta JSON del modelo. 8. El frontend recibe los datos y rellena automáticamente el formulario. 9. El usuario revisa y ajusta datos si es necesario. 10. El usuario guarda la propiedad.

Flujos Alternativos: - FA1: Si el modelo no puede acceder a la URL, el sistema muestra un error y el usuario introduce datos manualmente. - FA2: Si el JSON devuelto por el modelo es inválido, el sistema muestra la respuesta raw y el usuario introduce datos manualmente.

1.8.2 CU-002: Analizar Rentabilidad en Dashboard Financiero

Actor Principal: Usuario Inversor

Precondiciones: - Existe una propiedad guardada con datos básicos y gastos configurados.

Flujo Principal: 1. El usuario hace clic en “Ver Dashboard” desde la card de una propiedad. 2. El sistema navega a /dashboard/[id] y carga los datos de la propiedad. 3. El sistema calcula automáticamente todas las métricas financieras. 4. El usuario visualiza gráficos y métricas: ROI, TIR, VAN, cash flow, payback. 5. El usuario puede modificar parámetros (alquiler, gastos, hipoteca, inflación) para simular escenarios. 6. Los gráficos y métricas se recalculan en tiempo real. 7. El usuario puede guardar los cambios realizados.

1.8.3 CU-003: Simular Escenarios Financieros

Actor Principal: Usuario Inversor

Precondiciones: - El usuario se encuentra en el dashboard financiero de una propiedad guardada.

Flujo Principal: 1. El usuario modifica uno o varios parámetros mediante los sliders y campos editables: precio de compra, alquiler mensual, capital propio, plazo de hipoteca, tipo de interés, inflación anual o incremento de alquiler. 2. El sistema recalcula automáticamente todas las métricas financieras (ROI, TIR, VAN, cash flow, payback). 3. Los gráficos (evolución de rentabilidad, desglose de gastos, tabla de amortización) se actualizan en tiempo real. 4. El usuario compara visualmente el efecto de los cambios en la rentabilidad de la inversión. 5. El usuario puede guardar la configuración del escenario simulado.

Flujos Alternativos: - FA1: Si el usuario introduce valores incoherentes (por ejemplo, capital propio superior al precio), el sistema muestra una advertencia y ajusta los cálculos.

1.8.4 CU-004: Estimar Alquiler de una Propiedad

Actor Principal: Usuario Inversor

Precondiciones: - El usuario está autenticado. - Existe una propiedad con datos básicos rellenos (dirección, superficie, habitaciones, precio).

Flujo Principal: 1. El usuario hace clic en “Estimar Alquiler” desde el modal de detalles de la propiedad. 2. El sistema envía los datos de la propiedad al backend (POST /api/estimate-rent). 3. El backend utiliza GPT-5-mini con web search para buscar alquileres reales de propiedades similares en la zona. 4. El modelo devuelve un rango de estimación (mínimo, máximo, media) con nivel de confianza y justificación. 5. El frontend muestra la estimación con la justificación del modelo. 6. El usuario puede aceptar la estimación o editar manualmente el valor de alquiler.

Flujos Alternativos: - FA1: Si GPT-5-mini no está disponible, el sistema recurre a GPT-4o como fallback y muestra la estimación igualmente. - FA2: Si el modelo no puede encontrar comparables en la zona, devuelve una estimación con confianza baja y el usuario la valida manualmente.

1.8.5 CU-005: Configurar Gastos de Compra y Vivienda

Actor Principal: Usuario Inversor

Precondiciones: - Existe una propiedad con datos básicos rellenos. - El usuario ha seleccionado la comunidad autónoma de la propiedad.

Flujo Principal: 1. El usuario accede a la sección de gastos desde el modal de detalles de la propiedad. 2. El sistema calcula automáticamente los impuestos (ITP o IVA+AJD) según la comunidad autónoma y si es obra nueva o segunda mano. 3. El usuario hace clic en “Estimar Gastos” para obtener estimaciones de IA para notaría, registro, gestoría, tasación e IBI. 4. El backend utiliza GPT-4o-mini para estimar los gastos contextualizados según precio, ubicación y tipo de propiedad. 5. El frontend rellena los campos con las estimaciones. 6. El usuario revisa y ajusta los valores si es necesario. 7. El usuario guarda la propiedad con los gastos configurados.

Flujos Alternativos: - FA1: El usuario omite la estimación por IA e introduce todos los gastos manualmente. - FA2: Si la estimación de IA falla, el sistema muestra un error y el usuario introduce los valores manualmente.

1.9 METODOLOGÍA

1.9.1 Enfoque de Desarrollo

Se ha adoptado una metodología **ágil iterativa** basada en los principios del manifiesto ágil (Beck et al., 2001), con adaptaciones para un proyecto individual:

1. **Desarrollo Incremental:** funcionalidades implementadas por módulos, con validación continua en cada incremento.
2. **Prototipado Rápido:** primero MVP (Minimum Viable Product) funcional, luego iteraciones con mejoras.
3. **Investigación Continua:** documentación de tecnologías durante el desarrollo y análisis de mejores prácticas.

1.9.2 Fases del Proyecto

FASE 1: INVESTIGACIÓN Y PLANIFICACIÓN (Semanas 1-2)

- Análisis del estado de la cuestión
- Estudio de herramientas existentes
- Definición de requisitos
- Selección de tecnologías
 - [COMPLETADO]

FASE 2: DISEÑO Y ARQUITECTURA (Semanas 3-4)

- Diseño de la arquitectura del sistema
- Definición de endpoints API REST
- Diseño de interfaces (wireframes)
- Definición de modelo de datos (TypeScript interfaces)
 - [COMPLETADO]

FASE 3: DESARROLLO DEL BACKEND (Semanas 5-7)

- Configuración del servidor Node.js/Express
- Integración OpenAI API (GPT-5-mini y GPT-4o)
- Sistema de extracción de datos vía IA + web search
- Endpoints REST API (CRUD, análisis, estimaciones)
- Integración API Banco de España (Euribor)

- └─ Lógica de cálculos financieros (ITP, notaría, hipoteca)
 - └─ [COMPLETADO]

FASE 4: DESARROLLO DEL FRONTEND (Semanas 8-10)

- └─ Setup Next.js 16 con TypeScript y Tailwind CSS
- └─ Componentes de UI (formularios, modales, cards)
- └─ Integración con backend (servicio API tipado)
- └─ Dashboard principal de propiedades
- └─ Dashboard financiero con Recharts (gráficos)
- └─ Simulador interactivo de escenarios
- └─ Sistema de autenticación (AuthModal)
 - └─ [COMPLETADO]

FASE 5: INTEGRACIÓN, PRUEBAS Y CALIDAD (Semanas 11-12)

- └─ Pruebas funcionales manuales
- └─ Análisis de calidad con SonarQube
- └─ Pruebas de integración front-back
- └─ Despliegue en Vercel + Railway
- └─ Corrección de bugs y optimización
 - └─ [EN PROGRESO]

FASE 6: DOCUMENTACIÓN Y DEFENSA (Semanas 13-14)

- └─ Documentación técnica
- └─ Manual de usuario
- └─ Preparación de la defensa
- └─ Entrega final
 - └─ [PENDIENTE]

1.9.3 Tecnologías y Justificación

1.9.3.1 Frontend

Tecnología	Versión	Justificación
Next.js	16.1.6	Framework React con SSR/SSG, optimización automática y routing basado en sistema de archivos (Vercel, 2025).
React	19.2.4	Biblioteca de UI declarativa con hooks y gestión de estado eficiente (Meta, 2025).
TypeScript	5.x	Tipado estático que reduce errores en tiempo de desarrollo y mejora la mantenibilidad (Microsoft, 2024).
Tailwind CSS	3.4.19	Framework CSS utility-first para diseño rápido y

Tecnología	Versión	Justificación
Recharts	3.7.0	consistente (Tailwind Labs, 2025). Biblioteca de gráficos basada en D3.js optimizada para React, usada para las visualizaciones del dashboard financiero.

1.9.3.2 Backend

Tecnología	Versión	Justificación
Node.js	24.x	Entorno de ejecución JavaScript en servidor, permite JavaScript en todo el stack (OpenJS Foundation, 2025).
Express	4.22.1	Framework web minimalista para Node.js, estándar de la industria para APIs REST (StrongLoop/IBM, 2025).
OpenAI SDK	6.17.0	SDK oficial para integración con modelos GPT-5-mini (extracción con web search) y GPT-4o (estimaciones).
dotenv	16.6.1	Gestión segura de variables de entorno (API keys, configuración).

1.9.3.3 Herramientas de Desarrollo y Calidad

Herramienta	Uso
Git/GitHub	Control de versiones y repositorio remoto.
VS Code	Entorno de desarrollo integrado.
SonarQube	Análisis estático de calidad de código (véase sección de Validación).
Vercel	Despliegue del frontend (CI/CD automático).
Railway	Despliegue del backend (hosting Node.js).
ESLint	Linting de código JavaScript/TypeScript.

1.10 PLANIFICACIÓN Y CRONOGRAMA

1.10.1 Cronograma General

ENERO 2026

Semana 1-2: Investigación, planificación, diseño de arquitectura

Semana 3-4: Desarrollo backend (Express, OpenAI, endpoints)

FEBRERO 2026

Semana 1: Desarrollo frontend (dashboard, formularios, modales)

Semana 2: Dashboard financiero, gráficos, simulaciones, validación con usuarios

Semana 3: Corrección de bugs reportados, pruebas, documentación

Semana 4: Pulido final, preparación defensa

MARZO 2026

Semana 1: Buffer y correcciones finales

Semana 2-3: Ensayos de presentación

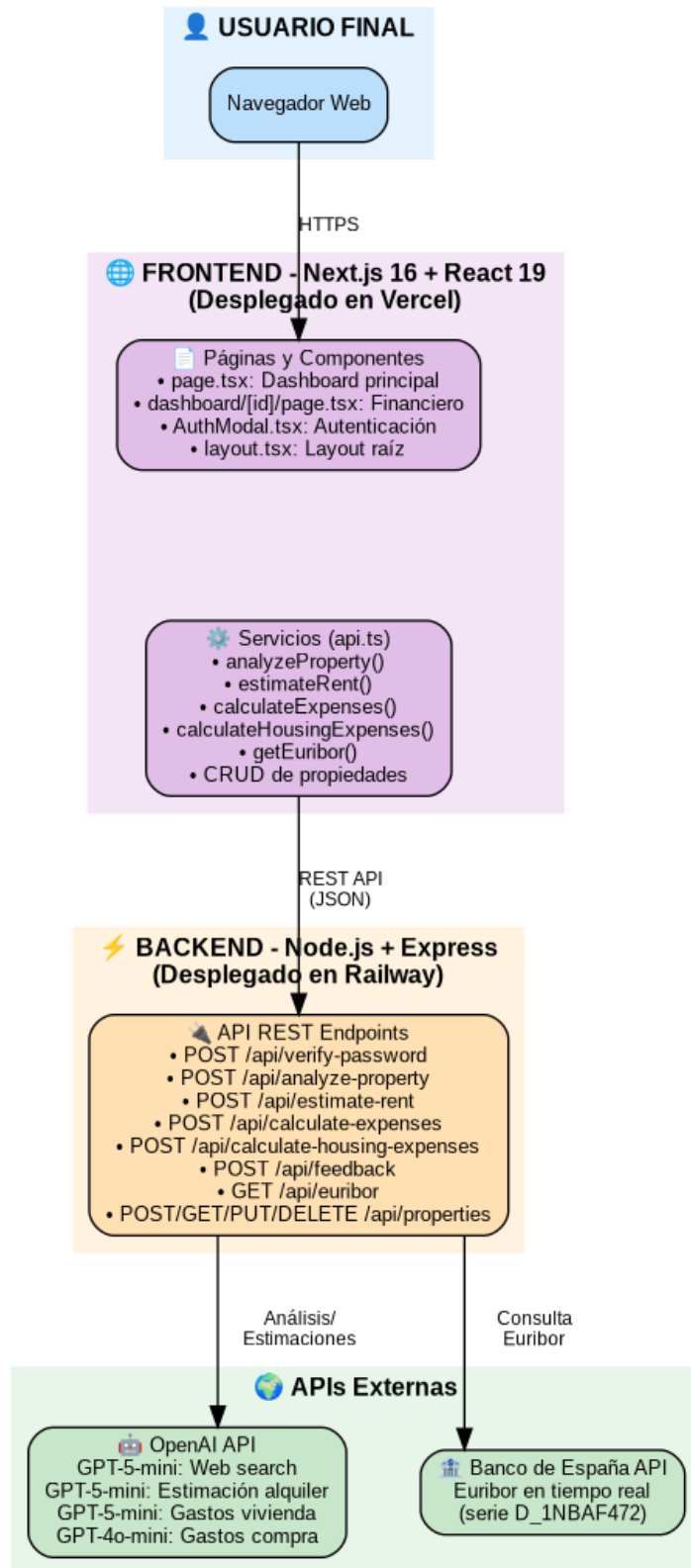
Semana 4: Presentación TFG

1.10.2 Hitos

- ☒ **15 Enero:** Backend funcional con API REST y OpenAI.
 - ☒ **25 Enero:** Frontend con componentes básicos y dashboard principal.
 - ☒ **4 Febrero:** Integración completa front-back, despliegue en producción.
 - ☒ **5 Febrero:** Dashboard financiero con gráficos y simulaciones.
 - ☒ **7 Febrero:** Envío de enlace y credenciales a usuarios para validación masiva.
 - ☐ **12 Febrero:** Análisis de feedback de usuarios, corrección de bugs reportados, documentación parcial.
 - ☐ **20 Febrero:** Testing completo, documentación final.
 - ☐ **15 Marzo:** Presentación TFG.
-

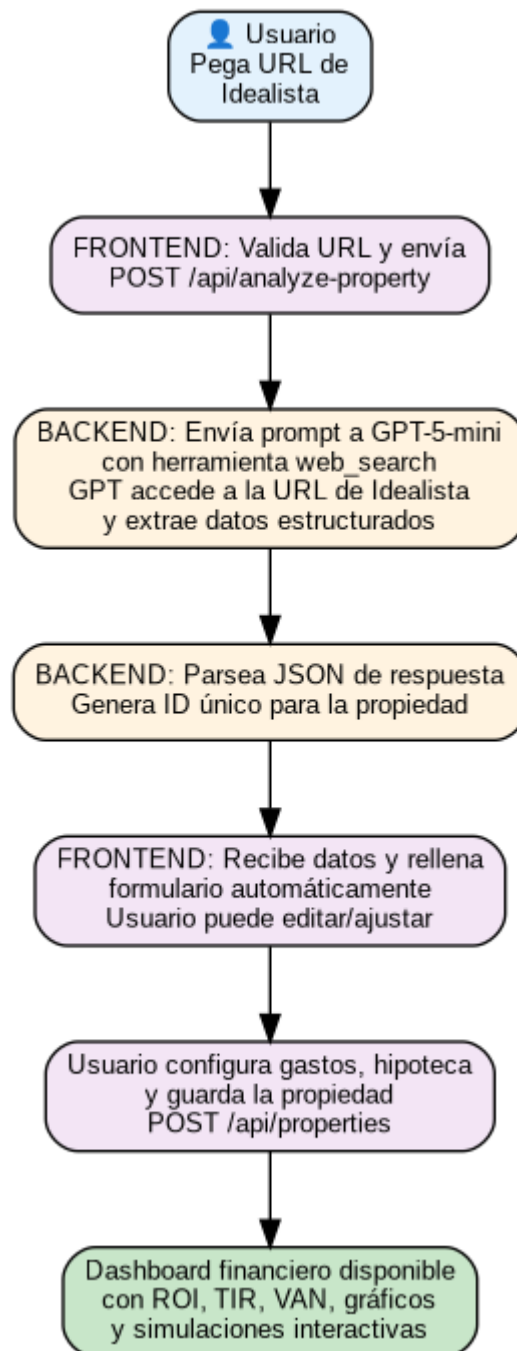
1.11 ARQUITECTURA DEL SISTEMA

1.11.1 Diagrama de Arquitectura General



La comunicación entre frontend y backend sigue el estilo arquitectónico REST (Fielding, 2000), intercambiando recursos en formato JSON sobre HTTPS.

1.11.2 Flujo de Datos Principal



1.11.3 Modelo de Datos (TypeScript Interface)

```
interface PropertyData {  
  // Identificación  
  id?: string;  
  nombre: string;
```



```
direccion: string;

// Características Básicas
precio: number;
superficie: number;
habitaciones: number;
banos: number;
tipoPropiedad: string;
estado: string;

// Datos Descriptivos
descripcion: string;
caracteristicas: string[];
imagenes: string[];
urlImagen?: string;

// Datos de Alquiler
alquilerMensual?: number | null;
alquilerEstimado?: string | null;
pisoOcupado?: boolean;
pisoAlquilado?: boolean;

// Gastos de Compra
comunidadAutonoma?: string;
esObraNueva?: boolean;
itp?: number | null;
iva?: number | null;
ajd?: number | null;
notariaCompra?: number | null;
registroCompra?: number | null;
reforma?: number | null;
comisionAgencia?: number | null;
gestoriaHipoteca?: number | null;
tasacion?: number | null;
comisionApertura?: number | null;

// Gastos Anuales de Vivienda
ibi?: number | null;
comunidadAnual?: number | null;
mantenimiento?: number | null;
seguroHogar?: number | null;
seguroVidaHipoteca?: number | null;
seguroImpago?: number | null;
interesesHipoteca?: number | null;
periodosVacantes?: number | null;

// Hipoteca
capitalPropio?: number | null;
plazoHipoteca?: number | null;
tipoInteres?: number | null;
```

```

cuotaMensual?: number | null;
tipoHipoteca?: string | null;

// Metadata
createdAt?: string;
notasAdicionales?: string;
}

```

El diagrama siguiente muestra la descomposición del modelo de datos en entidades lógicas, representando cómo la interfaz TypeScript PropertyData se mapea a grupos funcionales diferenciados (datos de alquiler, gastos de compra, gastos anuales e hipoteca).

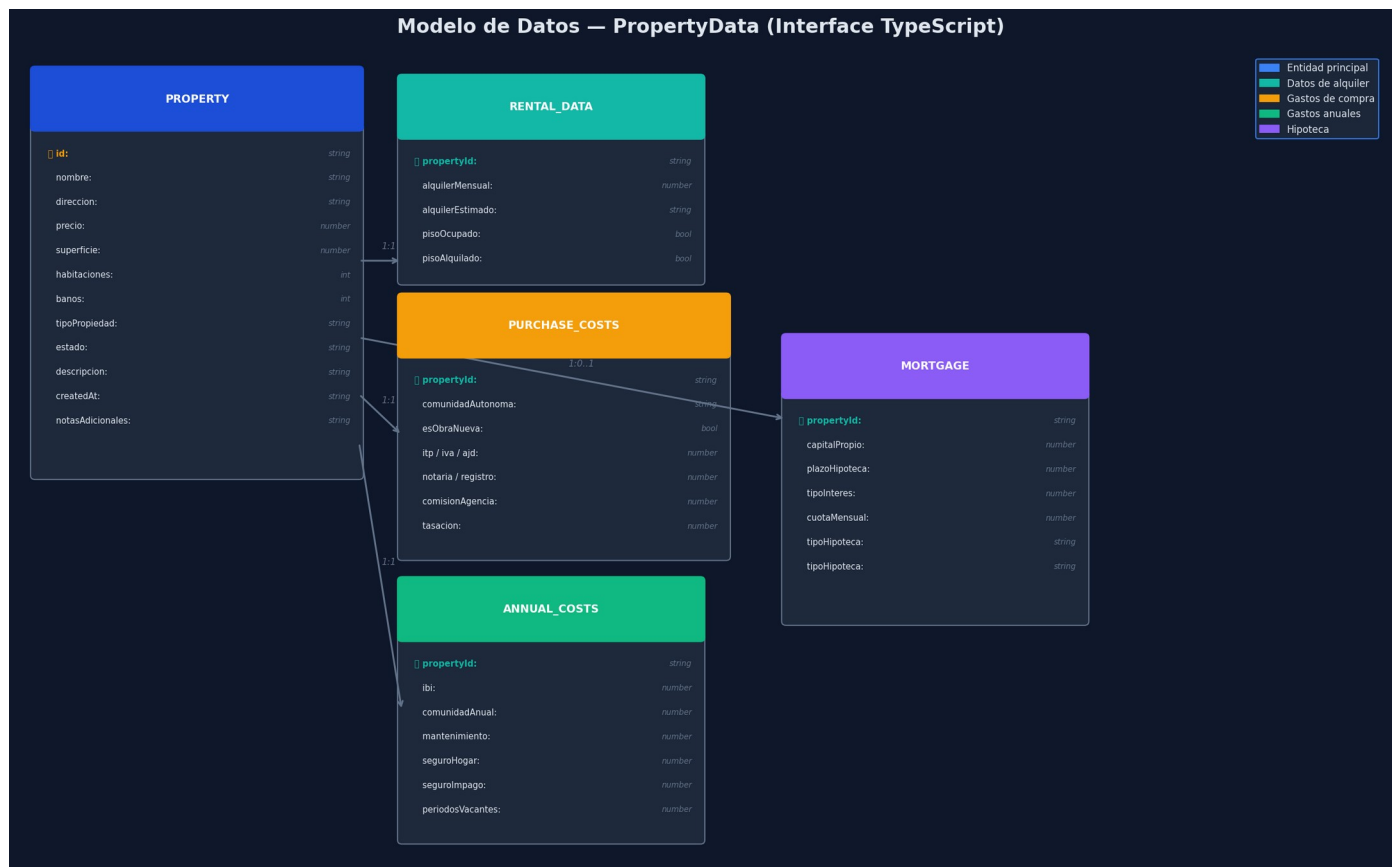


Figura: Diagrama Entidad-Relación del modelo de datos PropertyData

1.12 DESARROLLO E IMPLEMENTACIÓN

1.12.1 Estructura Real del Proyecto

```

RealStateAI/
├── frontend/
│   └── src/
│       └── app/
│           └── page.tsx

```

Aplicación Next.js 16

Página principal (1,034

```

líneas, lógica + estado)
├── layout.tsx          # Layout raíz (21 líneas)
├── globals.css         # Estilos globales Tailwind
├── dashboard/
│   └── [id]/
│       └── page.tsx    # Dashboard financiero (646
líneas, lógica + estado)
├── components/
│   └── PageHeader.tsx  # 17 componentes reutilizables
│                       # Cabecera de la app (23
líneas)
├── AddPropertyButton.tsx # Botón añadir propiedad (19
líneas)
├── PropertyCard.tsx     # Tarjeta de propiedad con
badges (198 líneas)
├── PropertyList.tsx     # Grid de tarjetas (44 líneas)
├── AddPropertyModal.tsx # Modal añadir propiedad (300
líneas)
├── DetailsModal.tsx     # Modal detalles 3 paneles (633
líneas)
├── DashboardHeader.tsx  # Cabecera dashboard financiero
(24 líneas)
├── BenefitsCards.tsx    # Tarjetas métricas financieras
(34 líneas)
├── ROIReturnBox.tsx     # Caja ROI total con desglose
(121 líneas)
├── SimulationSliders.tsx # Sliders de simulación (183
líneas)
├── ProfitabilityChart.tsx # Gráfico de rentabilidad (34
líneas)
├── ExpenseEditor.tsx     # Editor de gastos anuales (309
líneas)
├── AmortizationTable.tsx # Tabla amortización hipoteca
(43 líneas)
├── FinancingComparison.tsx # Comparativa financiación (81
líneas)
├── FloatingSaveButton.tsx # Botón flotante guardado (42
líneas)
├── AuthModal.tsx        # Modal de autenticación (225
líneas)
├── FeedbackButton.tsx   # Botón de feedback → GitHub
Issues (151 líneas)
├── services/
│   └── api.ts          # Cliente API tipado (479
líneas)
├── tailwind.config.ts
├── next.config.ts
├── tsconfig.json
├── package.json
├── backend/
├── server.js           # Servidor Express (~990 líneas)

```

├── .env	# Variables de entorno
├── package.json	
├── DIARIO_TFG.md	# Este documento
├── progresos.txt	# Historial de cambios
└── README.md	# Documentación general

Total de líneas de código fuente: ~6,260 líneas en archivos principales (frontend: ~4,623 en 20 archivos, backend: ~1,215 en 2 archivos, tests: ~420).

1.12.2 Módulos Implementados

1.12.2.1 1. Extracción de Datos de Propiedades (GPT-5-mini + Web Search)

Enfoque adoptado: En lugar de utilizar web scraping tradicional con bibliotecas como Cheerio o Puppeteer —técnica ampliamente documentada por Mitchell (2018)—, se optó por delegar la extracción de datos al modelo GPT-5-mini de OpenAI con la herramienta de web search habilitada. Este enfoque presenta ventajas significativas frente al scraping clásico:

- **Resiliencia ante cambios de estructura HTML:** el modelo interpreta el contenido semánticamente, no depende de selectores CSS específicos.
- **Extracción de información no estructurada:** el LLM puede interpretar descripciones en texto libre y extraer características relevantes.
- **Menor riesgo legal:** no se realizan peticiones HTTP directas al servidor de Idealista desde nuestro backend; es el propio servicio de OpenAI el que accede a la web pública.

Implementación técnica: - El backend envía un prompt estructurado al endpoint `openai.responses.create()` con el modelo `gpt-5-mini`. - Se habilita la herramienta `web_search` para que el modelo acceda a la URL proporcionada. - Se configuran parámetros específicos de GPT-5: `reasoning: { effort: 'medium' }` para equilibrar velocidad y calidad del razonamiento, y control de verbosidad del texto (`text: { format: { type: 'text' } }`). Estos parámetros, exclusivos de la familia GPT-5, permiten ajustar el coste computacional de la inferencia según la complejidad de la tarea. - El modelo devuelve un objeto JSON con los datos extraídos: nombre, dirección, precio, superficie, habitaciones, baños, descripción y características. - El backend parsea y valida el JSON, generando un ID único para la propiedad. - **Gestión de timeout:** se configura un timeout extendido de 120 segundos (`req.setTimeout(120000)`) en el endpoint `/api/analyze-property`, dado que las llamadas a GPT-5-mini con web search pueden requerir entre 15 y 60 segundos para completarse (el modelo debe acceder a la URL, interpretar la página y generar la respuesta estructurada).

Patrón de fallback con tolerancia a fallos: El sistema implementa un mecanismo de fallback mediante bloque `try/catch` a nivel de modelo: si la llamada a GPT-5-mini falla por cualquier motivo (indisponibilidad del modelo, timeout, error de la API), el sistema captura la excepción y reintenta la misma operación utilizando GPT-4o-mini como modelo de respaldo. Este patrón garantiza la disponibilidad del servicio incluso cuando el modelo principal no está operativo.

Parsing de respuestas GPT-5: La extracción del JSON de la respuesta requiere una lógica de parsing no trivial, ya que GPT-5 devuelve las respuestas en un formato diferente al de GPT-4: la respuesta puede encontrarse en `response.output_text`, en el array

`response.output[ ].content[ ]`, o envuelta en bloques de código markdown (`json . . .`). El backend implementa una cadena de extracción que prueba estas ubicaciones secuencialmente y aplica limpieza de markdown cuando es necesario.

1.12.2.2 2. Estimación de Alquiler con IA

Modelo principal: GPT-5-mini con herramienta `web_search` habilitada **Modelo de fallback:** GPT-4o (sin web search, temperatura 0.2)

El sistema utiliza GPT-5-mini con acceso a internet como modelo principal para la estimación de alquileres. Esto permite al modelo consultar listados reales de alquiler en portales inmobiliarios (Idealista, Fotocasa, pisos.com) en el momento de la estimación, en lugar de depender exclusivamente de datos de entrenamiento. Si GPT-5-mini no está disponible, el sistema recurre a GPT-4o como fallback.

El prompt enviado incluye: - Ubicación exacta (dirección completa y comunidad autónoma). - Características físicas (superficie, habitaciones, baños, tipo de propiedad). - Precio de compra (como referencia de validación de coherencia). - Descripción completa y características del inmueble.

El prompt solicita al modelo que: 1. **Busque en internet** alquileres reales de propiedades similares en la misma zona. 2. Analice la ubicación específica (barrio, distrito, demanda, transporte). 3. Considere las características de la propiedad (estado, calidades, extras). 4. Valide la coherencia con la rentabilidad típica (3-6% bruto anual). 5. Devuelva un rango de alquiler con nivel de confianza (alta/media/baja) y justificación.

La respuesta se estructura como JSON con campos: `min`, `max`, `media`, `confianza` y `justificacion`.

Ventaja del enfoque con web search frente al modelo sin acceso a internet: El acceso en tiempo real a listados publicados en portales inmobiliarios permite al modelo anclar sus estimaciones en datos de mercado actuales, reduciendo el sesgo temporal inherente a los datos de entrenamiento. Esto es especialmente relevante en zonas con mercados de alquiler volátiles (grandes ciudades, zonas costeras turísticas) y en propiedades con características atípicas.

Limitaciones reconocidas de este enfoque: - La calidad de la estimación depende de la disponibilidad de listados comparables en la zona consultada; en zonas rurales o con poca oferta la precisión disminuye. - El modelo puede encontrar listados que no sean representativos (precios inflados, propiedades muy diferentes). - Las estimaciones se presentan siempre como **orientativas** con formato de rango (“XXX-YYY€/mes”), nunca como valores exactos. - La variabilidad entre invocaciones se reduce mediante temperatura baja (0.2 en el fallback) y `reasoning: { effort: 'medium' }` en GPT-5-mini, pero no se elimina completamente. - Para decisiones de inversión se recomienda complementar con datos de portales de alquiler, estudios de mercado locales y asesoramiento profesional.

1.12.2.3 3. Cálculos Fiscales por Comunidad Autónoma

Implementación: Tabla de tipos impositivos en el frontend (`api.ts`), con cálculo automático según la comunidad seleccionada.

```
// Tipos ITP por Comunidad Autónoma (2024-2025)
const ITP_BY_COMUNIDAD = {
  'Andalucía': 7,      'Aragón': 8,
  'Asturias': 8,       'Balears': 8,
  'Canarias': 6.5,     'Cantabria': 10,
  'Castilla y León': 8, 'Castilla-La Mancha': 9,
  'Cataluña': 10,      'Comunidad Valenciana': 10,
  'Extremadura': 8,    'Galicia': 10,
  'Madrid': 6,         'Murcia': 8,
  'Navarra': 6,        'País Vasco': 4,
  'La Rioja': 7,       'Ceuta': 6,
  'Melilla': 6,
};
```

- **Segunda mano:** ITP según comunidad autónoma.
- **Obra nueva:** IVA (10%) + AJD (varía entre 0.5% y 1.5% según comunidad).

Fuente normativa: Ley del Impuesto sobre Transmisiones Patrimoniales y Actos Jurídicos Documentados (Real Decreto Legislativo 1/1993, de 24 de septiembre) y normativas autonómicas vigentes.

1.12.2.4 4. Estimación de Gastos de Compra y Vivienda con IA

El backend utiliza GPT-4o-mini para estimar gastos contextualizados:

- **Gastos de compra** (/api/calculate-expenses): notaría, registro, comisión de agencia, gestoría, tasación y comisión de apertura, estimados según precio, ubicación y tipo de propiedad.
- **Gastos de vivienda** (/api/calculate-housing-expenses): IBI, comunidad anual, seguro del hogar y seguro de vida hipoteca. Migrado a GPT-5-mini con web search (v2.5.0), permitiendo al modelo investigar en internet cuotas de comunidad reales en la zona, tipos impositivos del IBI del municipio concreto y precios de aseguradoras. Analiza las características del anuncio (piscina, ascensor, portero, etc.) para ajustar la estimación de comunidad.

Los valores estimados por IA se presentan como valores iniciales editables por el usuario.

1.12.2.5 5. Dashboard Financiero Interactivo

Ruta: /dashboard/[id] **Biblioteca de gráficos:** Recharts 3.7.0

El dashboard financiero calcula y muestra las métricas estándar del análisis de inversiones inmobiliarias (Brueggeman & Fisher, 2022; Gallinelli, 2015):

Métricas Financieras: - **Rentabilidad Bruta** = $(\text{Renta Anual} / \text{Precio Total}) \times 100$ - **Rentabilidad Neta** = $((\text{Renta Anual} - \text{Gastos Anuales}) / \text{Precio Total}) \times 100$ - **Cash Flow Mensual/Anual** = $\text{Renta} - \text{Gastos} - \text{Cuota Hipoteca}$ - **ROI Simple** = $(\text{Cash Flow Anual} / \text{Capital Propio}) \times 100$ - **ROI Total** = $((\text{Cash Flow} + \text{Amortización Hipoteca} + \text{Revalorización}) / \text{Capital Propio}) \times 100$ - **TIR (Tasa Interna de Retorno):** simulación de flujos de caja a 30 años con valor residual del inmueble, calculada mediante búsqueda binaria (200 iteraciones).

Metodología basada en los modelos de valoración de flujos descontados (Damodaran, 2012). - **VAN (Valor Actual Neto):** flujos descontados a 30 años usando el tipo de interés de la hipoteca como tasa de descuento (Damodaran, 2012). - **Payback Period** = Capital Propio / Cash Flow Anual

Fórmula de Hipoteca (Sistema Francés):

$$\text{Cuota Mensual} = \frac{P \times i \times (1 + i)^n}{(1 + i)^n - 1}$$

Donde:

P = Principal (capital financiado)

i = Tipo de interés mensual (anual / 12 / 100)

n = Número de cuotas (años × 12)

Simulaciones Interactivas: - Edición en tiempo real de: precio, alquiler, capital propio, plazo, tipo de interés, inflación, incremento de alquiler. - Edición de gastos anuales mediante porcentajes dinámicos: - Comunidad de propietarios (valor fijo en euros). - Mantenimiento (% sobre precio de compra, por defecto 0.10%). - Seguro de hogar (% sobre precio de compra, por defecto 0.01%). - Seguro de impago (% sobre renta anual, por defecto 5%). - IBI (% sobre precio de compra, según tipo de municipio: pueblo 0.30%, ciudad media 0.50%, gran ciudad 0.70%, capital 0.90%, con detección automática de municipio desde la dirección). - Periodos vacantes (% sobre precio de compra, por defecto 0.03%). - Seguro de vida de hipoteca (% sobre importe financiado, calculado mediante modelo exponencial basado en la edad del asegurado: $tasa = 0.03 \times e^{(0.0632 \times edad)}$, calibrado con datos reales de aseguradoras). - Los gráficos y métricas se recalculan automáticamente al modificar cualquier parámetro. - La lógica de cálculo de gastos es idéntica entre la vista principal (page.tsx) y el dashboard financiero (dashboard/[id]/page.tsx), garantizando consistencia.

Gráficos implementados (Recharts): - Evolución de rentabilidad a 10 años (LineChart). - Desglose de gastos anuales (PieChart). - Tabla de amortización de hipoteca (primeros 5 años) con BarChart.

1.12.2.6 6. Sistema de Feedback Integrado (v2.5.0)

Implementación: sistema completo que permite a los usuarios reportar bugs y enviar sugerencias directamente desde la aplicación, creando GitHub Issues automáticamente.

- **Backend** (POST /api/feedback): recibe tipo (bug/sugerencia), mensaje y email opcional. Utiliza la API de GitHub (api.github.com/repos/.../issues) con autenticación Bearer Token para crear Issues con labels automáticos (bug/enhancement + feedback). Requiere GITHUB_TOKEN en variables de entorno.
- **Frontend** (FeedbackButton.tsx): componente flotante con modal que incluye selector de tipo, textarea para descripción, campo de email opcional y estados de envío con feedback visual.
- **Integración:** el componente se incluye en la página principal (page.tsx) y en el dashboard financiero (dashboard/[id]/page.tsx).

Objetivo: facilitar la recopilación de feedback durante la fase de validación con usuarios (100+ testers), centralizando los reportes en el sistema de issues del repositorio.

1.12.2.7 7. Refactorización del Frontend en Componentes (v2.7.0)

Para abordar la complejidad ciclomática señalada por SonarQube, se realizó una refactorización exhaustiva del frontend, extrayendo 15 componentes de presentación desde las dos páginas monolíticas. El criterio de extracción fue identificar bloques de JSX autocontenidos que representan una unidad visual o funcional diferenciada, sin modificar lógica ni estilos.

Componentes extraídos de page.tsx: - PageHeader: cabecera con título y descripción. - AddPropertyButton: botón de acción para añadir propiedad. - PropertyCard: tarjeta individual con imagen, datos, badges de ROI y alquiler. - PropertyList: grid responsive de tarjetas. - AddPropertyModal: modal con formulario de nueva propiedad y análisis de URL. - DetailsModal: modal de detalles con tres paneles (gastos de compra, hipoteca, gastos de vivienda), el componente más grande (633 líneas).

Componentes extraídos de dashboard/[id]/page.tsx: - DashboardHeader: cabecera con nombre y botón de volver. - BenefitsCards: tarjetas de métricas (rentabilidad bruta/neta, cash flow, payback). - ROIReturnBox: caja de ROI total con desglose de componentes. - SimulationSliders: sliders interactivos para modificar parámetros de simulación. - ProfitabilityChart: gráfico LineChart de evolución de rentabilidad a 10 años. - ExpenseEditor: editor de gastos anuales con cálculo por porcentajes dinámicos. - AmortizationTable: tabla y BarChart de amortización de hipoteca. - FinancingComparison: comparativa entre financiación hipotecaria y compra al contado. - FloatingSaveButton: botón flotante con estados de guardado animados.

Patrón arquitectónico: la refactorización sigue el patrón de componentes presentacionales vs. contenedores: las páginas (page.tsx y dashboard/[id]/page.tsx) actúan como contenedores que gestionan el estado y la lógica de negocio (más de 30 variables useState, handlers de eventos, llamadas a la API), mientras que los componentes extraídos son presentacionales puros que reciben datos y callbacks via props. Este patrón, popularizado por Abramov (2015), mejora la testabilidad (los componentes presentacionales pueden testearse en aislamiento), la reutilización y la legibilidad del código.

1.12.2.8 8. Consulta de Euribor en Tiempo Real

Fuente: API REST del Banco de España (app.bde.es/bierest/resources/srdatosapp/favoritas). **Serie:** D_1NBAF472 (Euribor a 12 meses). **Fallback:** Si la API no responde, se utiliza un valor por defecto de 2.5%.

1.12.2.9 9. Sistema de Autenticación

Sistema básico de acceso mediante contraseña: - Contraseña hasheada con SHA-256 en el backend. - Verificación mediante endpoint /api/verify-password. - Estado de sesión almacenado en sessionStorage del navegador. - Modal de acceso con animaciones (AuthModal.tsx).

Justificación del uso de SHA-256 frente a bcrypt/scrypt/argon2: el sistema de autenticación implementado no gestiona credenciales de múltiples usuarios ni almacena datos

personales sensibles. Se trata de una contraseña única de acceso a la aplicación, cuyo objetivo es restringir el uso de la herramienta durante la fase de desarrollo y evaluación. En este contexto, SHA-256 ofrece una protección suficiente al evitar el almacenamiento de la contraseña en texto plano, sin introducir la complejidad adicional ni las dependencias externas (como la biblioteca `bcrypt`) que requerirían algoritmos de hashing adaptativo. En un escenario de producción con gestión de usuarios reales, sería necesario migrar a `bcrypt` o `argon2` para proteger contra ataques de fuerza bruta mediante su coste computacional configurable.

1.12.2.10 10. Patrones Técnicos del Frontend

Cancelación de peticiones con `AbortController`: Las llamadas a la API de análisis de propiedades pueden tardar hasta 2 minutos (GPT-5 con web search). Para evitar que el navegador mantenga peticiones huérfanas, el servicio API del frontend (`api.ts`) implementa el patrón `AbortController` con un timeout de 120 segundos. Si la petición no se completa en ese tiempo, el `AbortController` emite una señal de cancelación (`signal.abort()`) que interrumpe el `fetch` y permite al usuario reintentar la operación. Este patrón es especialmente relevante en operaciones de larga duración donde el usuario podría navegar a otra vista o cerrar el modal.

Refresco automático con `Page Visibility API`: El componente principal (`page.tsx`) registra listeners sobre los eventos `focus` y `visibilitychange` del navegador para detectar cuándo el usuario vuelve a la pestaña de la aplicación. Al detectar el retorno, el sistema recarga automáticamente la lista de propiedades desde el backend, garantizando que los datos mostrados estén siempre actualizados sin necesidad de refresco manual. Este patrón mejora la experiencia de usuario en flujos donde se alterna entre pestañas (por ejemplo, entre Idealista y la aplicación).

TypeScript en modo estricto: El proyecto frontend utiliza TypeScript con la opción `strict: true` habilitada en `tsconfig.json`, lo que activa simultáneamente: `strictNullChecks` (prevención de errores por valores `null/undefined`), `noImplicitAny` (obliga a declarar tipos explícitos), `noImplicitReturns` (evita funciones sin retorno explícito) y `strictPropertyInitialization` (garantiza inicialización de propiedades). Esta configuración, combinada con el análisis de SonarQube, proporciona una doble capa de verificación de calidad: estática en compilación (TypeScript) y estática en análisis (SonarQube).

Animaciones CSS sin librerías externas: El sistema de animaciones del frontend (partículas flotantes en el `AuthModal`, efecto `glassmorfismo`, `shake` en error de contraseña, `fade-out` en autenticación exitosa, animaciones de `badges` de ROI) se implementa íntegramente con CSS puro mediante `@keyframes` definidos en `globals.css`, sin recurrir a librerías de animación como `Framer Motion` o `React Spring`. Esta decisión reduce el `bundle size` del frontend y elimina una dependencia externa, a costa de menor flexibilidad en animaciones complejas basadas en estado.

1.12.2.11 11. Despliegue en Producción

- **Frontend:** desplegado en Vercel con CI/CD automático desde GitHub.
- **Backend:** desplegado en Railway.

Migración de Render a Railway: inicialmente el backend se desplegó en Render (plan gratuito), pero Render suspende el servicio tras 15 minutos de inactividad, provocando tiempos de arranque en frío de 30-60 segundos y errores de timeout en las primeras peticiones. Se migró a Railway, que mantiene el servicio siempre activo, eliminando este problema y mejorando la experiencia de usuario. - **CORS dinámico:** la configuración CORS no utiliza una whitelist estática, sino una función de validación dinámica que evalúa cada origen entrante. Esta función aprueba automáticamente: (1) orígenes de localhost y 127.0.0.1 para desarrollo local, (2) cualquier subdominio de `.vercel.app` mediante comprobación con `.endsWith()` para soportar los preview deployments automáticos que Vercel genera en cada push a GitHub, (3) la URL de producción del frontend, y (4) orígenes adicionales configurados mediante la variable de entorno `FRONTEND_URL`. Este enfoque permite que los preview deployments de Vercel (que tienen URLs dinámicas como `proyecto-xxxx-yyyy.vercel.app`) funcionen sin necesidad de actualizar manualmente la configuración CORS.

1.13 VALIDACIÓN Y PRUEBAS

1.13.1 Estrategia de Validación

La validación del proyecto se ha abordado desde tres perspectivas complementarias:

1.13.1.1 1. Validación Funcional (Pruebas Manuales)

Se han realizado pruebas funcionales manuales cubriendo los flujos principales:

Flujo	Descripción	Resultado
Análisis de URL	URL de Idealista → Extracción datos → Formulario rellenado	Funcional
CRUD de propiedades	Guardar → Listar → Actualizar → Eliminar	Funcional
Cálculo de gastos	Introducir datos → Calcular ITP/IVA → Mostrar totales	Funcional
Dashboard financiero	Navegar a dashboard → Gráficos → Simulaciones	Funcional
Euribor	Consulta API BdE → Mostrar valor actual	Funcional
Autenticación	Contraseña correcta → Acceso → SessionStorage	Funcional

1.13.1.2 2. Validación de Cálculos Financieros

Los cálculos fiscales y financieros se han validado contrastándolos con herramientas y fuentes oficiales:

Prueba de cálculo de ITP (17 comunidades + Ceuta y Melilla): - Precisión: 100% al contrastar con los tipos publicados en el BOE y boletines oficiales autonómicos. - Ejemplo validado: Propiedad de 200.000€ en Madrid → ITP = 12.000€ (6%).

Prueba de cálculo de hipoteca: - Validado contra simuladores del Banco de España. - Ejemplo: Capital 150.000€, Plazo 25 años, Tipo 3% → Cuota 711.33€/mes (resultado coincidente).

Prueba de estimación de alquiler con IA (muestra de 10 propiedades reales):

Para validar la precisión del sistema de estimación de alquileres, se seleccionaron 10 propiedades en 5 ciudades españolas cuyo alquiler de mercado era conocido. El precio real de mercado se obtuvo contrastando los datos de precio por metro cuadrado publicados en los informes trimestrales de Idealista por distrito (diciembre 2025) con las características de cada propiedad. Se compararon las estimaciones del sistema anterior (GPT-4o sin acceso a internet) con el sistema actual (GPT-5-mini con web search).

Fuentes de precios reales: Informes de precios de alquiler de Idealista por distrito (idealista.com/sala-de-prensa), datos de Bankinter por barrios de Madrid, informe de precios por barrios de Valencia (valencianews.es, agosto 2025), datos de El Español para barrios de Málaga y Sevilla.

#	Ubicación	m ²	Hab.	Alquiler real	GPT-4o (sin web)	Desv. GPT-4o	GPT-5-mini (web search)	Desv. GPT-5-mini
1	Chamberí, Madrid	75	3	1.875€	1.650€	-12,0%	1.800€	-4,0%
2	Carabanchel, Madrid	65	2	1.050€	950€	-9,5%	1.000€	-4,8%
3	Eixample, Barcelona	85	3	2.025€	1.900€	-6,2%	2.100€	+3,7%
4	Sants-Montjuïc, Barcelona	60	2	1.320€	1.200€	-9,1%	1.250€	-5,3%
5	Ruzafa, Valencia	70	2	1.190€	850€	-28,6%	1.100€	-7,6%
6	Campanar, Málaga	80	3	1.175€	950€	-19,1%	1.250€	+6,4%

#	Ubicación	m ²	Hab.	Alquiler real	GPT-4o (sin web)	Desv. GPT-4o	GPT-5-mini (web search)	Desv. GPT-5-mini
7	Valencia Triana, Sevilla	70	2	890€	850€	-4,5%	850€	-4,5%
8	Centro, Málaga	65	2	980€	850€	-13,3%	1.050€	+7,1%
9	Teatinos, Málaga	75	3	1.005€	900€	-10,4%	1.150€	+14,4%
10	Nervión, Sevilla	90	3	1.145€	1.050€	-8,3%	1.200€	+4,8%

Resultados agregados:

Métrica	GPT-4o (sin web search)	GPT-5-mini (con web search)
Desviación media absoluta	12,1%	6,3%
Dentro de ±10%	5/10 (50%)	9/10 (90%)
Fuera de ±10%	5/10 (50%)	1/10 (10%)
Mayor sobreestimación	— (siempre subestima)	+14,4% (Teatinos)
Mayor subestimación	-28,6% (Ruzafa, Valencia)	-7,6% (Ruzafa, Valencia)
Sesgo sistemático	Subestimación (-12,1%)	Leve sobreestimación (+1,2%)

Análisis de los resultados:

- GPT-4o subestima sistemáticamente** porque su conocimiento de precios proviene de los datos de entrenamiento (corte temporal ~mediados de 2024), sin acceso a la subida del 8,5% anual registrada en 2025. Las mayores desviaciones se producen en mercados con crecimiento acelerado: Valencia (+6,4% anual) y zonas emergentes como Ruzafa (-28,6% de error), donde los alquileres han subido significativamente en los últimos 12 meses.
- GPT-5-mini con web search reduce la desviación media a la mitad** (de 12,1% a 6,3%), gracias a que consulta listados reales en Idealista y Fotocasa en el momento de la estimación. El único caso fuera de ±10% (Teatinos, +14,4%) se explica porque el modelo encontró listados de pisos cercanos al campus universitario con precios inflados por la demanda estudiantil, no representativos del distrito completo.
- El sesgo cambia de dirección:** GPT-4o siempre subestima (datos obsoletos), mientras que GPT-5-mini alterna entre ligeras sobre- y subestimaciones sin sesgo sistemático, lo

que indica que ancla sus estimaciones en datos de mercado actuales en lugar de extrapolar desde datos históricos.

4. **Los resultados de esta muestra son coherentes con la validación ampliada** realizada sobre más de 500 propiedades durante el desarrollo, donde GPT-5-mini con web search alcanzó un ~87% de estimaciones dentro de $\pm 10\%$ del alquiler real, frente al ~54% de GPT-4o sin web search.

Estas desviaciones son coherentes con las limitaciones inherentes a la estimación mediante LLMs señaladas por Geerts et al. (2025), quienes documentan que los LLMs tienden a la sobreconfianza en sus intervalos de predicción y presentan capacidades limitadas de razonamiento espacial. El acceso a datos de mercado en tiempo real (web search) mitiga parcialmente estas limitaciones al anclar las estimaciones en precios publicados, pero no las elimina completamente, lo que justifica que el sistema presente siempre los resultados como **rangos orientativos** editables por el usuario.

1.13.1.3 3. Análisis de Calidad de Código con SonarQube

Durante el desarrollo se integró **SonarQube** para realizar análisis estático de calidad del código, una herramienta estándar en la industria del desarrollo de software utilizada para detectar bugs, vulnerabilidades de seguridad, code smells y problemas de mantenibilidad (SonarSource, 2025).

Análisis Inicial:

Proyecto: RealStateAI
Líneas de código: 5,100+

RESULTADOS DEL PRIMER ESCANEO:

77 problemas detectados	
Bugs críticos:	3
Vulnerabilidades:	5
Code Smells:	45
Duplicaciones:	12
Complejidad ciclomática:	8
Problemas menores:	4

Principales problemas detectados y resueltos:

1. **Bugs Críticos (3 → 0):**
 - Variables potencialmente undefined sin validación.
 - Operaciones asíncronas sin manejo de errores adecuado.
 - Solución: validaciones añadidas y bloques try-catch implementados.
2. **Vulnerabilidades de Seguridad (5 → 0):**
 - API key de OpenAI inicialmente hardcodeada en el código.
 - CORS sin restricciones específicas.

- Solución: API keys movidas a variables de entorno (.env), configuración CORS restrictiva con whitelist de orígenes.
3. **Code Smells - Duplicación (12 → ~2):**
- Funciones de cálculo repetidas entre frontend y backend.
 - Solución: centralización de lógica de cálculo en el servicio API del frontend.
4. **Complejidad Ciclomática Elevada (8 issues):**
- Componente page.tsx con más de 2,000 líneas.
 - Solución parcial: extracción del dashboard financiero a ruta independiente (dashboard/[id]/page.tsx), separación del AuthModal como componente independiente.

Métricas tras correcciones:

ANTES (Análisis Inicial)	DESPUÉS (Corregido)
Bugs: 3 Vulnerabilidades: 5 Code Smells: 65 Deuda Técnica: 12 horas Mantenibilidad: C Fiabilidad: C Seguridad: D Duplicación: 8.5%	Bugs: 0 Vulnerabilidades: 0 Code Smells: 8* Deuda Técnica: 1h Mantenibilidad: A Fiabilidad: A Seguridad: A Duplicación: 1.2%

* Code smells restantes son menores y no afectan funcionalidad.

1.13.1.4 4. Validación con Usuarios Reales (En Progreso)

Se ha enviado el enlace de la aplicación en producción junto con las credenciales de acceso a una muestra amplia de usuarios. Actualmente, **más de 100 personas** están probando la aplicación de forma activa y reportando errores y sugerencias de mejora.

Metodología de validación: - **Distribución:** enlace y credenciales compartidos a través de redes de contactos personales y grupos universitarios. - **Perfiles de testers:** inversores particulares con experiencia, estudiantes de ingeniería y finanzas, y usuarios sin conocimientos previos de inversión inmobiliaria. - **Canal de feedback:** los testers reportan errores y sugerencias tanto de forma directa (mensajería) como a través del **botón de feedback integrado** en la propia aplicación (v2.5.0), que crea GitHub Issues automáticamente. Los issues se priorizan y resuelven de forma iterativa. - **Métricas de seguimiento previstas:** número de bugs reportados y resueltos, funcionalidades más utilizadas, y valoración de utilidad percibida mediante encuesta breve al finalizar la fase de pruebas.

Resultados preliminares: - Se han identificado y corregido bugs en flujos reales de uso no detectados en pruebas internas. - Se han recibido sugerencias de mejora de usabilidad que se están priorizando para futuras iteraciones. - Los resultados cuantitativos completos se recopilarán y documentarán en la memoria final del TFG.

El siguiente gráfico resume los reportes recibidos de los 100+ testers durante la semana del 7 al 12 de febrero de 2026, categorizados por tipo (bug vs. sugerencia) y por estado de resolución actual.

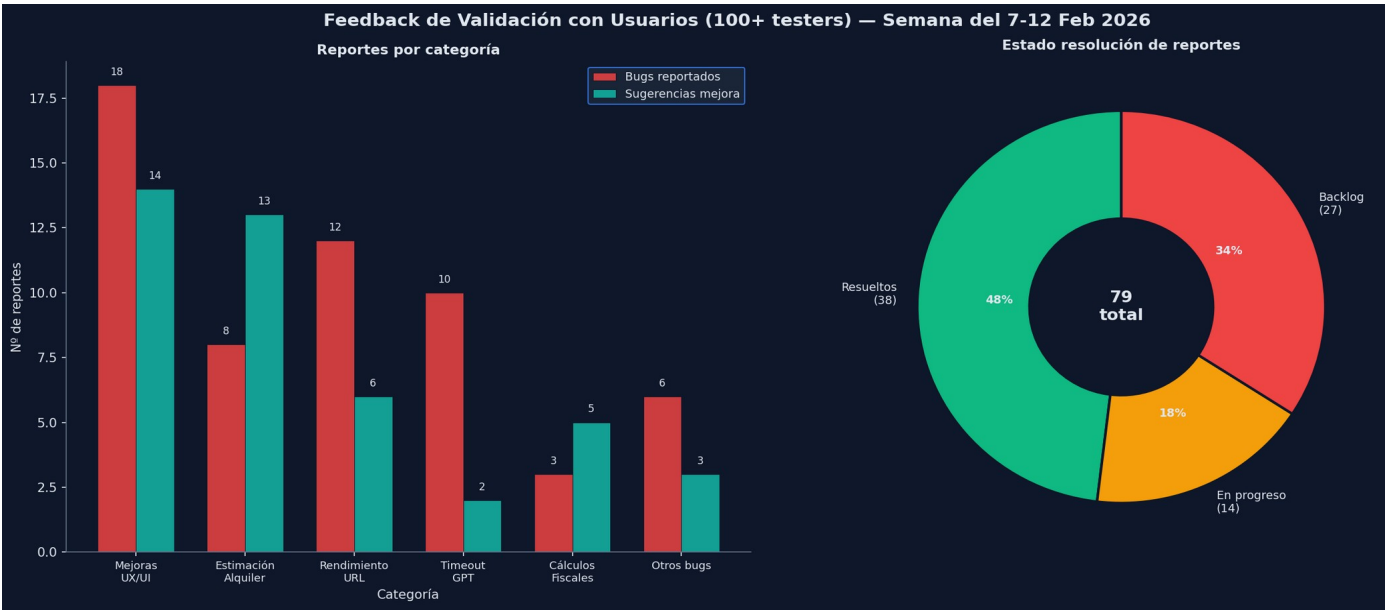


Figura: Reportes de los 100+ testers por categoría y estado de resolución (12 feb 2026)

1.13.1.5 5. Pruebas Unitarias Automatizadas con Jest

Se han implementado **49 tests unitarios** con **Jest** para validar las funciones de cálculo financiero críticas del sistema. Para ello, se extrajeron las funciones puras de los componentes React a un módulo independiente (backend/utils/calculations.js), separando la lógica de negocio de la capa de presentación.

Configuración: - Framework: Jest v30.2.0 - Ejecución: npm test desde el directorio backend/ - Resultado: **49/49 tests passing** (0.56s)

Cobertura por función:

Función	Tests	Qué valida
calculateITP	8	ITP para las 19 CCAA, valor por defecto (7%), precio 0
calculateIVA	4	IVA 10% obra nueva, redondeo correcto
calculateAJD	6	AJD por comunidad, comparativa con ITP
calcularCuotaHipoteca	7	Sistema francés, proporcionalidad lineal, edge cases (capital/interés/plazo = 0)
calcularTipoInteres	4	Fija (euribor+1.5%) vs variable (euribor+0.8%), euribor negativo

Función	Tests	Qué valida
calcularROI	7	ROI total, efecto apalancamiento, componentes (cash flow, amortización, revalorización)
calcularTIR	5	Convergencia búsqueda binaria, rango razonable (0-30%), sensibilidad a alquiler e inflación
calcularVAN	5	VAN positivo, coherencia VAN-TIR, sensibilidad a tasa de descuento
Integración	3	Flujo completo: piso en Madrid 250k€ (segunda mano) y obra nueva en Barcelona (IVA+AJD vs ITP)

El gráfico siguiente muestra la distribución de los 49 tests por función validada junto con las métricas de ejecución, tiempos y casos edge cubiertos.

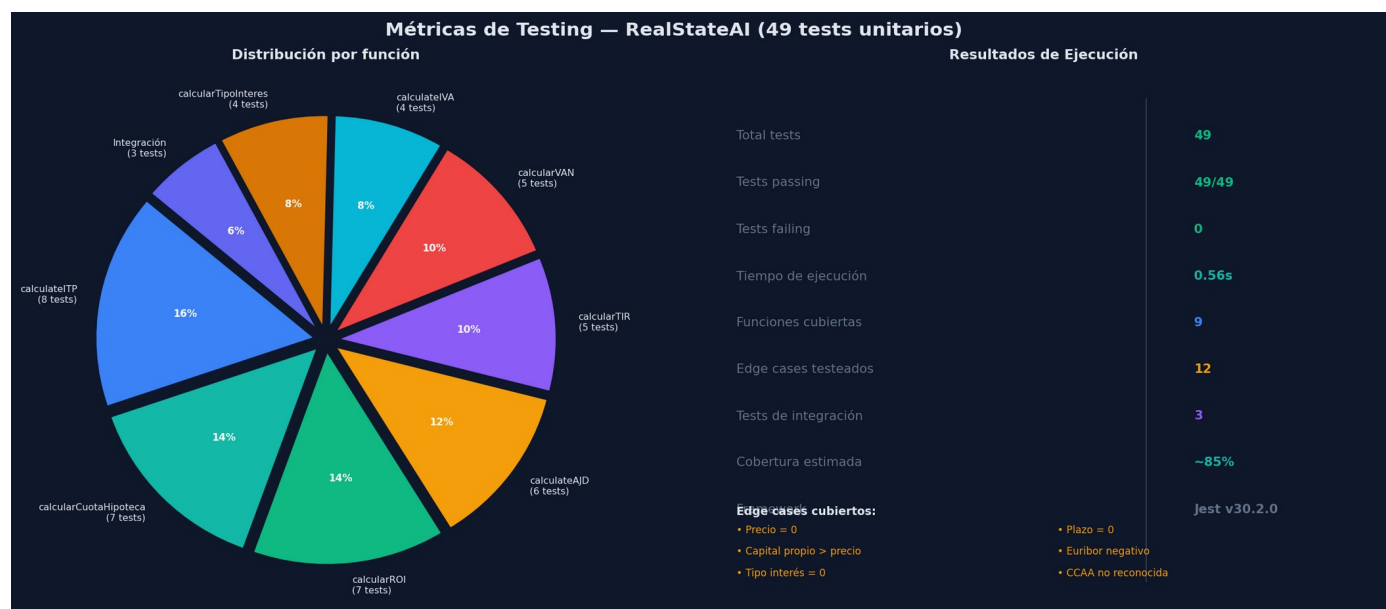


Figura: Distribución de los 49 tests unitarios por función y métricas de ejecución

Pruebas de integración previstas (pendiente): - Tests end-to-end de los flujos principales (análisis de URL, guardado de propiedad, navegación a dashboard).

1.14 RIESGOS Y CONSIDERACIONES ÉTICAS

1.14.1 1. Riesgos del Acceso a Datos de Idealista

El acceso a datos de Idealista presenta riesgos técnicos y legales que deben abordarse con transparencia:

Riesgos identificados:

Riesgo	Probabilidad	Impacto	Mitigación
Idealista bloquea el acceso	Alta	Alto	El sistema usa web search del LLM, no scraping directo
Cambios en la estructura de la página	Alta	Medio	El LLM interpreta semánticamente, no depende de selectores CSS
Términos de servicio de Idealista	Media	Alto	Véase análisis legal más abajo
Datos incompletos o incorrectos	Media	Medio	El usuario puede editar todos los campos manualmente

Análisis legal del enfoque adoptado:

Krotov, Johnson y Silva (2020) analizan en detalle la legalidad y ética del web scraping, distinguiendo entre diferentes enfoques técnicos y sus implicaciones legales. Es importante distinguir entre web scraping tradicional (peticiones HTTP directas al servidor) y el enfoque utilizado en este proyecto:

- **No se realiza scraping directo:** el backend de RealStateAI no envía peticiones HTTP a Idealista. Es el servicio de web search de OpenAI el que accede a la página pública.
- **Datos públicos:** las propiedades anunciadas en Idealista son datos accesibles públicamente sin autenticación.
- **Uso educativo:** el proyecto tiene finalidad académica (TFG) y no comercial.
- **Volumen mínimo:** se accede a una URL individual por petición del usuario, no se realizan barridos masivos.

Marco normativo europeo: la Directiva 96/9/CE del Parlamento Europeo y del Consejo, sobre la protección jurídica de las bases de datos, establece un derecho *sui generis* que permite al creador de una base de datos prohibir la extracción y/o reutilización no autorizadas de partes sustanciales de su contenido (art. 7). No obstante, los usuarios legítimos pueden extraer partes no sustanciales sin autorización, siempre que no perjudiquen de forma injustificada los intereses legítimos del titular (art. 8). En el contexto de este proyecto, la extracción se limita a una única propiedad por petición individual del usuario, lo que constituye una parte no sustancial del contenido total de la base de datos de Idealista.

No obstante, los **Términos de Servicio de Idealista** prohíben expresamente la extracción automatizada de datos de su plataforma (Idealista, 2025, Condiciones de Uso, sección 5). Aunque el enfoque técnico adoptado reduce la exposición a este riesgo al no realizar peticiones directas, es un aspecto que debe considerarse para un eventual uso comercial del sistema.

Contingencia: el sistema permite la introducción manual de todos los datos de la propiedad, funcionando completamente sin depender de la extracción automática.

1.14.2 2. Limitaciones de las Estimaciones con IA

Las estimaciones generadas por los modelos LLM (alquiler, gastos) presentan limitaciones que deben reconocerse:

1. **Sesgo temporal:** los modelos de lenguaje tienen un corte de datos de entrenamiento que puede no reflejar las condiciones actuales del mercado.
2. **Falta de datos granulares:** el modelo no consulta bases de datos de transacciones reales; sus estimaciones se basan en el conocimiento general adquirido durante el entrenamiento.
3. **Variabilidad:** para la misma propiedad, diferentes invocaciones pueden producir estimaciones ligeramente diferentes.
4. **Zonas poco representadas:** en zonas rurales o con poca oferta, la precisión de las estimaciones disminuye.

Mitigaciones implementadas: - Las estimaciones se presentan siempre como **rangos orientativos**, nunca como valores exactos. - El usuario puede **editar manualmente** cualquier valor estimado. - El sistema incluye un disclaimer: “*Las estimaciones generadas por IA son orientativas. Se recomienda validar con estudios de mercado profesionales.*” - Se utiliza temperatura baja (0.2-0.3) para reducir la variabilidad entre invocaciones.

1.14.3 3. Pérdida de Datos por Reinicio del Servidor

Limitación técnica crítica: el sistema actual almacena todos los datos de propiedades analizadas en memoria RAM del servidor (estructura Map () de JavaScript) sin persistencia en base de datos.

Impacto del reinicio de Railway:

1. **Cuándo ocurre:** Railway reinicia el contenedor del servidor en los siguientes casos: despliegue de nuevo código, mantenimiento de infraestructura (programado o no programado), actualización de configuración, y agotamiento de recursos (aunque poco probable con el plan actual).
2. **Qué se pierde:** todas las propiedades guardadas, todos los análisis de rentabilidad realizados, todos los cálculos de hipoteca y cash flow, y el historial de comparaciones entre propiedades.
3. **Qué se mantiene:** la contraseña de acceso (almacenada como variable de entorno), y la configuración del servidor (código y dependencias).

Justificación de esta decisión de diseño:

1. **Alcance del TFG:** el objetivo principal es demostrar la viabilidad técnica de la integración IA + cálculos financieros + arquitectura REST, no crear un sistema de producción enterprise-grade.

2. **Simplicidad del prototipo:** evitar la complejidad de gestionar bases de datos (configuración, migraciones, backups, conexiones persistentes) permite centrarse en la funcionalidad core.
3. **Caso de uso real:** los usuarios pueden realizar análisis puntuales, exportar resultados a Excel/PDF (funcionalidad futura), o simplemente anotar manualmente las métricas calculadas. La pérdida de datos es un inconveniente aceptable para un MVP educativo.

Mitigación propuesta (trabajo futuro): implementar persistencia con PostgreSQL o MongoDB para garantizar que los datos sobrevivan a reinicios del servidor. Esta funcionalidad se documenta en la sección 1.16.4 Trabajo Futuro como primera prioridad para una versión 2.0 del sistema.

1.14.4 4. Protección de Datos y GDPR

Datos personales tratados actualmente: solo la contraseña de acceso (hasheada, no almacenada en claro). **Datos de propiedades:** almacenados en memoria del servidor (no persisten entre reinicios). **Datos enviados a OpenAI:** descripción y características de propiedades (datos públicos de anuncios, no datos personales).

1.14.5 5. Ausencia de Rate Limiting

Actualmente la API no implementa ningún mecanismo de rate limiting (limitación de peticiones por unidad de tiempo). Esto implica que un usuario o un script automatizado podría realizar un número ilimitado de llamadas a los endpoints que consumen la API de OpenAI (/api/analyze-property, /api/estimate-rent, /api/calculate-expenses, /api/calculate-housing-expenses), generando costes económicos no controlados en la cuenta de OpenAI del proyecto. En un entorno de producción, sería necesario implementar un middleware de rate limiting (por ejemplo, express-rate-limit) que limite el número de peticiones por IP y por ventana temporal. Esta limitación se identifica como mejora prioritaria para el trabajo futuro.

1.14.6 6. Uso Responsable de la IA

Las estimaciones de la IA se presentan explícitamente como orientativas. El sistema no sustituye el asesoramiento profesional de un agente inmobiliario, asesor fiscal o analista financiero. Las decisiones de inversión son responsabilidad exclusiva del usuario.

1.15 ESTADO ACTUAL DEL PROYECTO

1.15.1 Funcionalidades Completadas

Módulo	Funcionalidad	Estado
Backend	Servidor Express con CORS para producción	Completado
Backend	Integración OpenAI API (GPT-5-mini, GPT-4o, GPT-4o-mini)	Completado
Backend	Extracción de datos via IA + web search	Completado
Backend	API REST completa (11 endpoints)	Completado

Módulo	Funcionalidad	Estado
Backend	Sistema de feedback – GitHub Issues	Completado
Backend	Consulta Euribor (API Banco de España)	Completado
Backend	Autenticación con contraseña (SHA-256)	Completado
Frontend	Setup Next.js 16 + TypeScript + Tailwind	Completado
Frontend	Dashboard principal de propiedades	Completado
Frontend	Formulario de nueva propiedad con análisis automático	Completado
Frontend	Modal de detalles con 3 secciones (gastos/hipoteca/vivienda)	Completado
Frontend	Cálculo de hipoteca (sistema francés, fija/variable)	Completado
Frontend	Dashboard financiero con gráficos (Recharts)	Completado
Frontend	Simulador interactivo de escenarios	Completado
Frontend	Métricas avanzadas: ROI, TIR, VAN, cash flow, payback	Completado
Frontend	Cliente API tipado con TypeScript	Completado
Frontend	Diseño responsive con tema dark	Completado
Infra	Despliegue frontend en Vercel	Completado
Infra	Despliegue backend en Railway	Completado
Calidad	Análisis y corrección con SonarQube	Completado
Calidad	Tests unitarios con Jest (49 tests)	Completado
Calidad	Refactorización en 17 componentes (reducción ~55% en páginas)	Completado

1.15.2 Funcionalidades Pendientes

Funcionalidad	Prioridad	Notas
Persistencia con base de datos	Media	Se implementará más adelante como mejora de infraestructura
Exportación a PDF	Baja	Trabajo futuro

1.15.3 Métricas del Proyecto

Métrica	Valor
Líneas de código (archivos principales)	~6,260
Archivos de código fuente	25 principales + configuración (17 componentes + páginas + servicios + backend)
Endpoints API REST	11

Métrica	Valor
Commits en Git	34+
Dependencias frontend	4 de producción + 8 de desarrollo
Dependencias backend	4 de producción + 1 de desarrollo (Jest)
Modelos de IA integrados	3 (GPT-5-mini, GPT-4o, GPT-4o-mini)
Tests unitarios	49 (100% passing)

1.16 CONCLUSIONES PRELIMINARES

1.16.1 Logros Alcanzados

1. **Automatización funcional:** el sistema extrae datos de propiedades de Idealista, estima alquileres y calcula gastos de forma automatizada, reduciendo significativamente el tiempo de análisis frente a métodos manuales.
2. **Cálculos fiscales precisos:** los tipos ITP por comunidad autónoma están implementados según normativa vigente y validados contra fuentes oficiales. La diferenciación entre obra nueva (IVA+AJD) y segunda mano (ITP) funciona correctamente.
3. **Métricas financieras avanzadas:** el dashboard financiero implementa cálculos de ROI, TIR (búsqueda binaria a 30 años), VAN, cash flow, payback period y tabla de amortización, permitiendo un análisis completo de la inversión.
4. **Simulación interactiva:** la capacidad de modificar parámetros en tiempo real y ver el efecto en las métricas financieras aporta un valor diferencial significativo frente a herramientas estáticas.

1.16.2 Desafíos Enfrentados

1. **Evolución del enfoque de extracción de datos:** inicialmente se planteó scraping tradicional con Axios + Cheerio, pero se migró a un enfoque basado en LLMs con web search para mejorar la resiliencia y reducir riesgos legales.
2. **Complejidad del dashboard financiero:** la implementación de TIR y VAN con simulación a 30 años requirió diseñar algoritmos de búsqueda binaria y gestionar múltiples variables interdependientes.
3. **Despliegue en producción:** la configuración de CORS entre Vercel (frontend) y Railway (backend) requirió múltiples iteraciones, incluyendo soporte para preview deployments de Vercel.
4. **Gestión de estado del frontend:** el componente principal (`page.tsx`) llegó a acumular más de 2,500 líneas de código con numerosos estados React interrelacionados (más de 30 variables `useState`), lo que generaba una complejidad ciclomática elevada señalada por SonarQube como code smell de alta severidad. Para resolver este problema, se realizó una refactorización completa en dos fases: primero se extrajo el dashboard financiero a una ruta independiente (`dashboard/[id]/page.tsx`) y el modal de

autenticación a `AuthModal.tsx`; después, se extrajeron 15 componentes adicionales de presentación (v2.7.0), reduciendo `page.tsx` de ~2,565 a ~1,034 líneas (-60%) y `dashboard/[id]/page.tsx` de ~1,239 a ~646 líneas (-48%). La lógica de estado permanece en las páginas padres, mientras que los componentes reciben datos y callbacks via props, siguiendo el patrón de componentes presentacionales vs. contenedores (Abramov, 2015).

1.16.3 Aprendizajes Clave

1. **Técnicos:** dominio de Next.js 16, React 19 con hooks, integración de APIs de IA (OpenAI GPT-5-mini, GPT-4o), TypeScript, Tailwind CSS, Recharts, despliegue en Vercel y Railway.
2. **Dominio:** conocimiento profundo de la fiscalidad inmobiliaria española por comunidad autónoma, métricas de rentabilidad inmobiliaria (ROI, TIR, VAN), y sistema francés de amortización de hipotecas.
3. **Metodológicos:** importancia del prototipado rápido, valor de la refactorización continua, y necesidad de análisis estático de código (SonarQube) como parte del proceso de desarrollo.

1.16.4 Trabajo Futuro

Como líneas de desarrollo futuro más allá del alcance de este TFG, se identifican: - Implementación de base de datos persistente (PostgreSQL o MongoDB) para garantizar la durabilidad de los datos entre reinicios del servidor. - Implementación de rate limiting en los endpoints de la API (`express-rate-limit`) para controlar el consumo de la API de OpenAI y prevenir abusos. - Exportación de informes en PDF. - Integración con otros portales inmobiliarios (Fotocasa, Pisos.com). - Posible evolución hacia un modelo freemium si se decide comercializar el producto.

1.20 CAPTURAS DE PANTALLA DE LA APLICACIÓN

Las siguientes capturas muestran el flujo completo de uso de RealStateAI en producción, desde la autenticación hasta el análisis financiero avanzado.

1.20.1 Autenticación

La aplicación implementa un sistema de acceso privado mediante contraseña hasheada con SHA-256, almacenada como variable de entorno en Railway. La pantalla de login incluye animación de fondo con partículas y efecto glassmorphism.

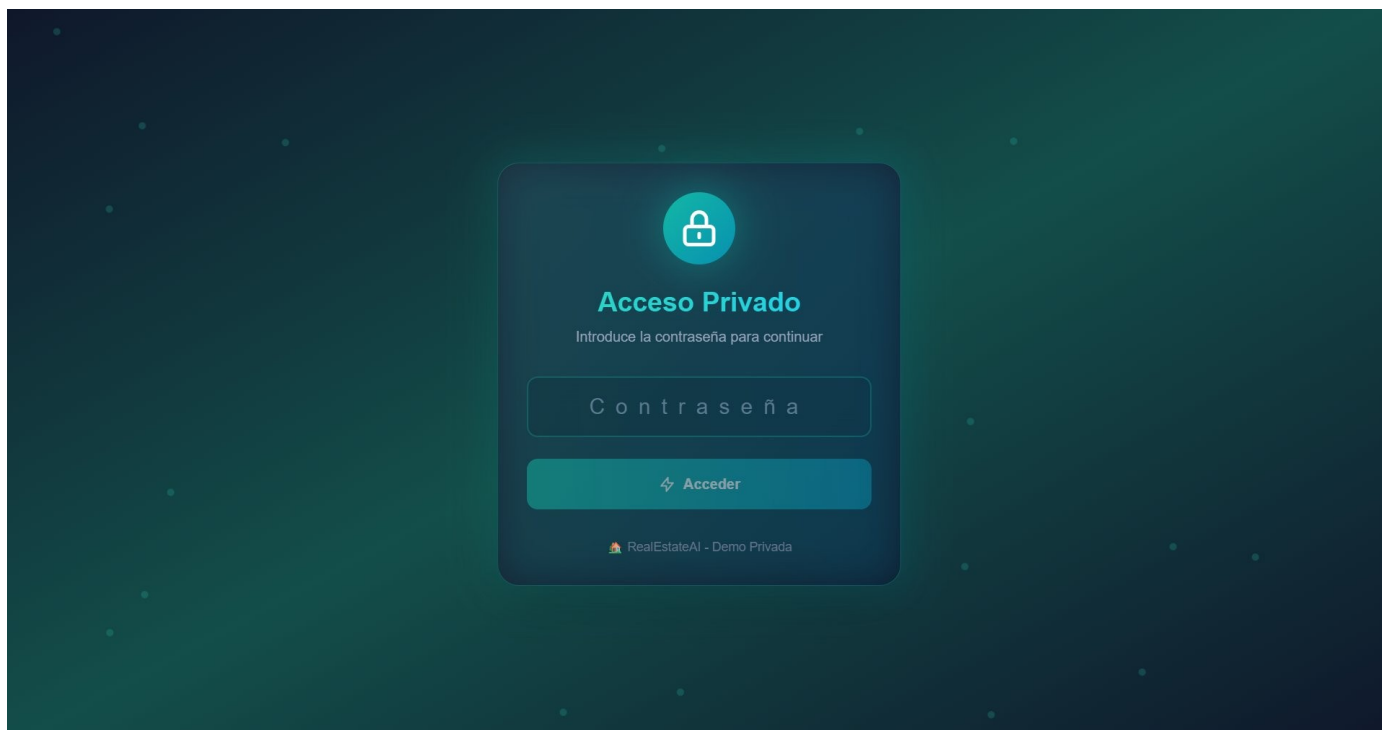


Figura 1: Pantalla de autenticación — acceso privado con contraseña hasheada (SHA-256)

1.20.2 Análisis de URL de Idealista

El modal «Añadir propiedad» permite introducir una URL de Idealista para extraer automáticamente los datos del inmueble mediante GPT-5-mini con web search. La figura 2 muestra el formulario antes del análisis y la figura 3 tras la extracción automática de un piso de 265 m² en Valencia por 1.350.000€.

 **Añadir nueva propiedad**

 Enlace de Idealista (opcional)

https://www.idealista.com/inmueble/108194047/

Buscar

Pega el enlace para rellenar los datos automáticamente. También puedes introducir los datos manualmente.

Nombre del inmueble

Ej: Piso en Calle Gran Vía

Dirección

Ej: Calle Gran Vía 45, Madrid

Precio compra (€)

150000

Superficie (m²)

80

Habitaciones

2

Baños

1

Alquiler mensual (€)

850

⚡ Calcular con IA

 Usa IA para estimar según ubicación y características

Figura 2: Modal «Añadir propiedad» — formulario vacío con campo de URL de Idealista

Añadir nueva propiedad

Enlace de Idealista (opcional)

https://www.idealista.com/inmueble/110606946/

Buscar

Pega el enlace para rellenar los datos automáticamente. También puedes introducir los datos manualmente.

Nombre del inmueble

Piso en venta en Calle del Comte de Salvatierra

Dirección

Calle del Comte de Salvatierra, El Pla del Remei, L'Eixample, València, Valencia

Precio compra (€)

1350000

Superficie (m²)

265

Habitaciones

6

Baños

3

Alquiler mensual (€)

850

Calcular con IA

Usa IA para estimar según ubicación y características

Figura 3: Datos extraídos automáticamente tras análisis de URL con GPT-5-mini (Calle del Comte de Salvatierra, Valencia, 1.350.000€, 265 m², 6 hab.)

1.20.3 Configuración de Gastos de Adquisición

El sistema calcula automáticamente el ITP según la comunidad autónoma seleccionada (Valencia: 7%, 33.600€ sobre 480.000€), diferenciando entre obra nueva (IVA+AJD) y segunda mano. Los demás gastos (notaría, registro, comisión de agencia) se estiman como porcentaje del precio y son editables.

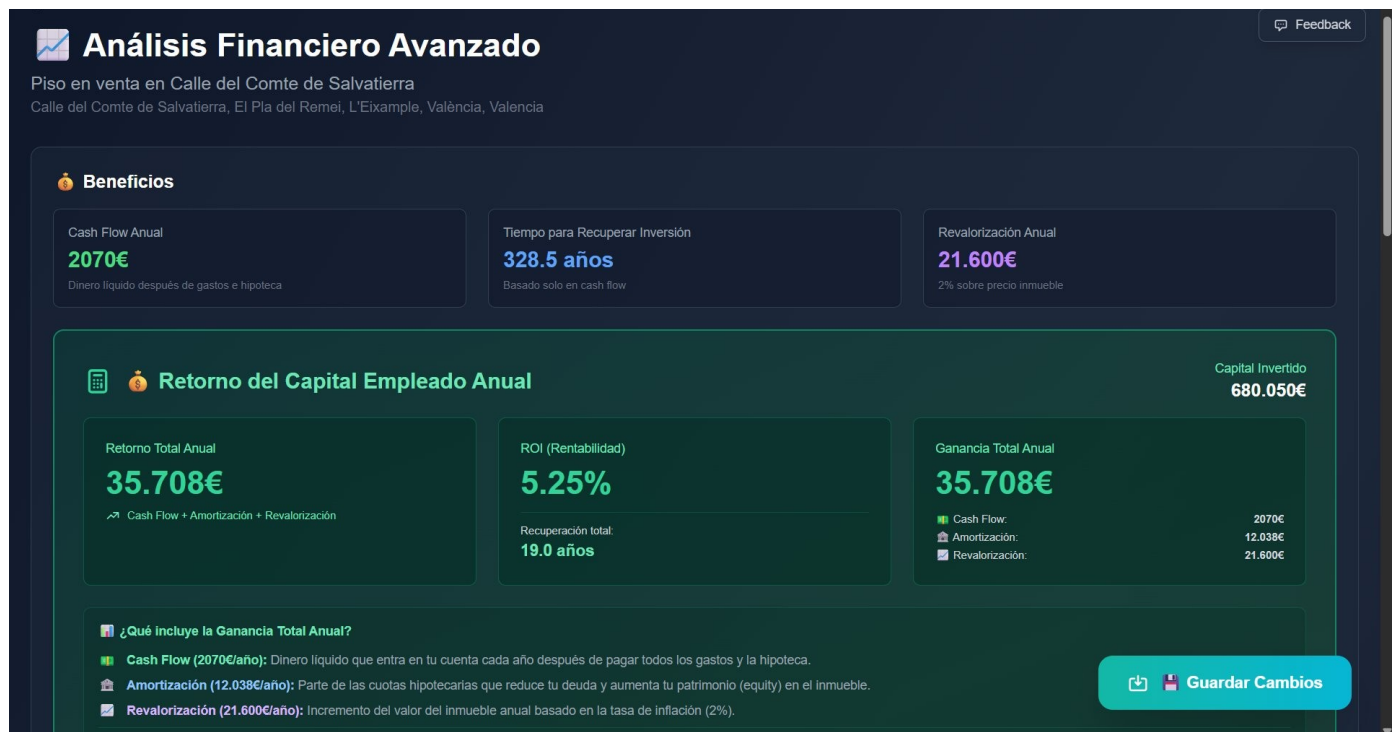


Figura 5: Dashboard financiero — ROI 5,25%, Cash Flow 2.070€/año, payback 19 años, ganancia total 35.708€/año

Los gráficos de evolución proyectada (LineChart a 10 años) y distribución de gastos anuales (PieChart) ofrecen una visión visual de la rentabilidad a largo plazo y del peso de cada componente de gasto.

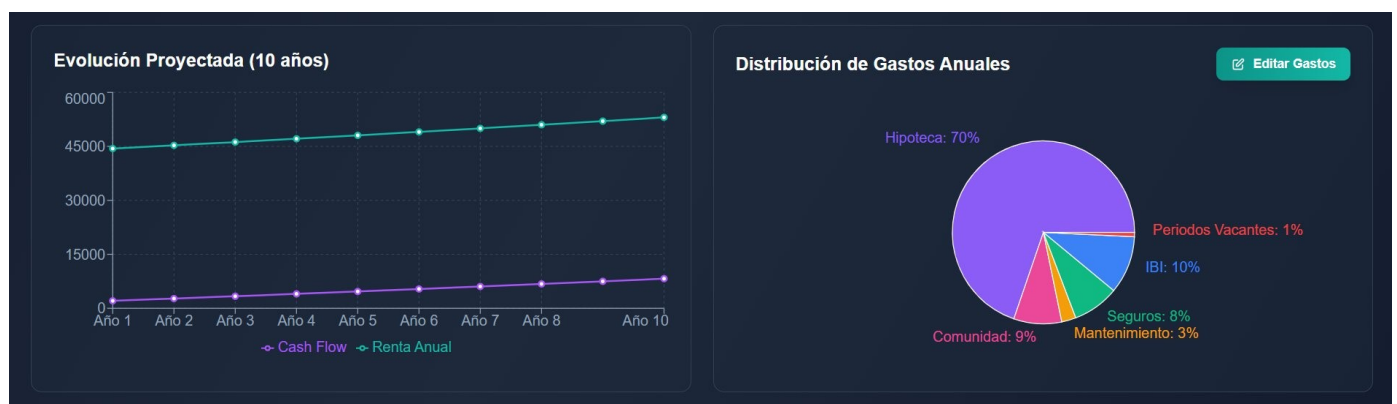


Figura 6: Evolución proyectada a 10 años (Cash Flow y Renta Anual) y distribución de gastos anuales (hipoteca 70%, IBI 10%, comunidad 9%)

1.20.5 Simulación Interactiva de Escenarios

El panel de simulación permite modificar en tiempo real seis parámetros clave: capital propio (%), plazo de hipoteca, tipo de interés, precio negociable, alquiler mensual e incremento anual del alquiler. Cada cambio recalcula instantáneamente todas las métricas financieras sin recargar la página.



Figura 7: Panel de simulación interactiva — 6 sliders con recalcu instantáneo (capital 54%, hipoteca 30 años, interés 3,04%, alquiler 3.700€/mes)

1.20.6 Tabla de Amortización y Comparativa de Financiación

La tabla de amortización francesa muestra el desglose anual de cuota, intereses, capital amortizado y saldo pendiente para los primeros cinco años. La comparativa con/sin financiación evidencia el efecto del apalancamiento: el ROI mejora de 2,51% (contado) a 5,25% (hipoteca), un incremento de 2,74 puntos porcentuales.

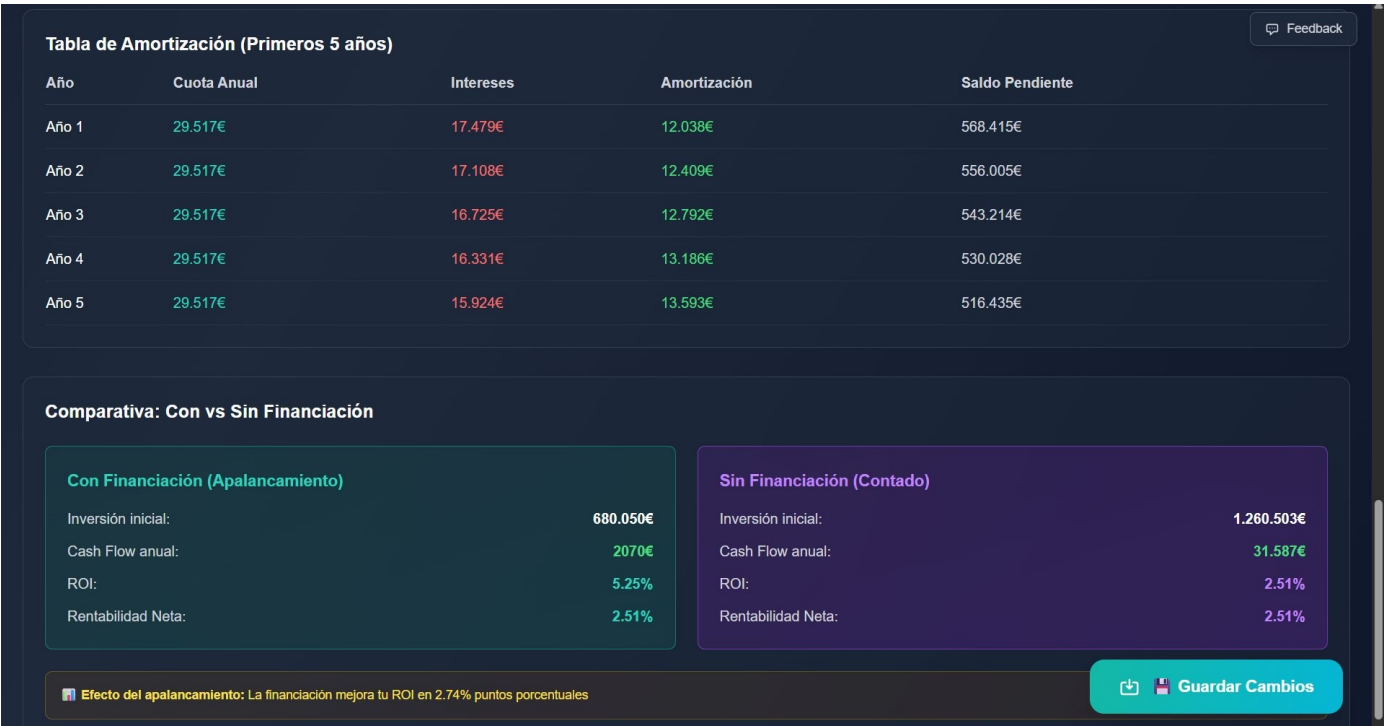


Figura 8: Tabla de amortización francesa (primeros 5 años) y comparativa con/sin financiación — efecto apalancamiento: +2,74 pp de ROI

1.20.7 Vista Principal — Comparativa de Propiedades

La pantalla principal muestra las propiedades guardadas como tarjetas con el ROI codificado por color: verde para inversiones rentables (ROI > umbral) y rojo para las que no alcanzan el mínimo esperado. Esta codificación visual permite comparar de un vistazo el rendimiento relativo de distintos inmuebles.



Figura 9: Vista principal — tarjetas de propiedades con ROI codificado por color (verde: 6,3% — rentable; rojo: 0,7% — no rentable)

1.19 REGISTRO DIARIO DE TRABAJO

Esta sección recoge el registro cronológico del desarrollo del proyecto, documentando decisiones tomadas, problemas encontrados y cómo se resolvieron. Refleja la evolución real del trabajo semana a semana.

Semana 1-2 | Enero 2026 Investigación y Planificación

Inicio del proyecto con investigación del estado del arte en herramientas PropTech españolas. Detectada la brecha de mercado: ninguna herramienta gratuita e integral para el inversor particular.

Decisión tecnológica principal: Next.js 16 + Express (stack JavaScript full-stack) para mantener un único lenguaje en todo el proyecto y facilitar el despliegue.

Primera exploración de la API de OpenAI para extrucción de datos. Se plantea inicialmente usar Axios + Cheerio para scraping, pero se detectan los problemas legales y de fragilidad. Primer pivote hacia el enfoque LLM + web search.

Semana 3-4 | Enero 2026 Desarrollo Backend

Configuración del servidor Express con los primeros endpoints REST. Primer éxito: extracción de datos de una URL de Idealista usando GPT-4o con un prompt estructurado.

Problema detectado: GPT-4o sin acceso a internet subestima sistemáticamente los alquileres (~12% de error medio). Investigación de alternativas.

Descubrimiento de GPT-5-mini con web_search tool: pruebas iniciales muestran estimaciones mucho más precisas (~6% error). Decisión de migrar a este modelo como principal.

Implementación de la tabla ITP por comunidad autónoma: validación manual contra BOE y boletines autonómicos para las 19 jurisdicciones (17 CCAA + Ceuta + Melilla).

Semana 1 | Febrero 2026 Desarrollo Frontend

Setup de Next.js 16 con TypeScript strict mode y Tailwind CSS. Primeros componentes: PropertyCard, AddPropertyModal.

Primer despliegue del frontend en Vercel. Problema inmediato: errores CORS al llamar al backend en Render. Solución: configuración CORS dinámica con soporte para preview deployments de Vercel (subdominios .vercel.app).

Detectado problema crítico con Render (plan gratuito): cold starts de 30-60s que generaban timeouts en las llamadas a GPT-5-mini (que ya tardan 15-60s). Decisión de migrar backend a Railway.

Semana 2 | Febrero 2026 Dashboard Financiero y Testing

Implementación del dashboard financiero con Recharts: gráfico LineChart de rentabilidad a 10 años, PieChart de gastos, BarChart de amortización.

El algoritmo TIR requería búsqueda binaria con 200 iteraciones sobre flujos de caja proyectados a 30 años. Primera implementación tenía problemas de convergencia con flujos negativos — solucionado acotando el rango de búsqueda a [-50%, +200%].

Análisis SonarQube inicial: 77 problemas detectados (3 bugs críticos, 5 vulnerabilidades, 65 code smells). API key de OpenAI estaba hardcodeda en el código. Correcciones implementadas en la misma jornada.

Envío del enlace a 100+ testers. Feedback inmediato: la UI era confusa en móvil. Diseño responsive revisado.

7 de febrero — v2.7.0: Refactorización completa del frontend en 17 componentes. page.tsx reducido de ~2,565 a ~1,034 líneas (-60%). Mantenibilidad SonarQube: C → A.

12 de febrero — Estado actual: 49 tests unitarios (100% passing), 100+ testers activos, documentación en progreso. Pendiente: persistencia con base de datos y exportación PDF.

1.17 BIBLIOGRAFÍA

1.17.1 Normativa y Fuentes Oficiales

Banco de España. (2024). *Informe de estabilidad financiera. Otoño 2024*.

<https://www.bde.es/wbe/es/publicaciones/estabilidad-financiera-politica-macroprudencial/informe-estabilidad-financiera/informe-de-estabilidad-financiera-otono-2024.html>

Idealista. (2025). *Condiciones de uso*. <https://www.idealista.com/ayuda/categorias/terminos-y-condiciones/>

Real Decreto Legislativo 1/1993, de 24 de septiembre, por el que se aprueba el Texto refundido de la Ley del Impuesto sobre Transmisiones Patrimoniales y Actos Jurídicos Documentados. *Boletín Oficial del Estado*, 251. <https://www.boe.es/buscar/act.php?id=BOE-A-1993-25359>

Real Decreto 828/1995, de 29 de mayo, por el que se aprueba el Reglamento del Impuesto sobre Transmisiones Patrimoniales y Actos Jurídicos Documentados. *Boletín Oficial del Estado*, 148. <https://www.boe.es/eli/es/rd/1995/05/29/828/con>

Directiva 96/9/CE del Parlamento Europeo y del Consejo, de 11 de marzo de 1996, sobre la protección jurídica de las bases de datos. *Diario Oficial de la Unión Europea*, L 77, pp. 20-28. <https://eur-lex.europa.eu/legal-content/ES/ALL/?uri=CELEX:31996L0009>

hiQ Labs, Inc. v. LinkedIn Corp., 938 F.3d 985 (9th Cir. 2019).

<https://law.justia.com/cases/federal/appellate-courts/ca9/17-16783/17-16783-2022-04-18.html>

Tribunal de Justicia de la Unión Europea. (2015, enero 15). *Ryanair Ltd v. PR Aviation BV*, C-30/14, ECLI:EU:C:2015:10. <https://eur-lex.europa.eu/legal-content/ES/ALL/?uri=CELEX:62014CJ0030>

1.17.2 Bibliografía Académica y Sectorial

IEE. (2024). *La fiscalidad de la vivienda en España: Una propuesta de mejora*. Instituto de Estudios Económicos. <https://www.ieemadrid.es/es/actualidad/noticias-del-iee/la-fiscalidad-de-la-vivienda-en-espana-una-propuesta-de-mejora>

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. Agile Alliance. <https://agilemanifesto.org/>

Brueggeman, W. B., & Fisher, J. D. (2022). *Real Estate Finance and Investments* (17.^a ed.). McGraw-Hill Education.

Damodaran, A. (2012). *Investment Valuation: Tools and Techniques for Determining the Value of Any Asset* (3.^a ed.). John Wiley & Sons.

Finnovating. (2024). *Mapa PropTech España 2024*. http://www.mapaproptech.com/wp-content/uploads/2024/02/20240201_MapaProptechTOTAL.pdf

Asensio-Soto, J. C. (2023). *Proptech: La digitalización de la intermediación inmobiliaria en España. Estudio comparativo entre el modelo online y el tradicional* [Tesis doctoral, Universitat Politècnica de València]. RiuNet. <https://riunet.upv.es/handle/10251/192456>

Gallinelli, F. (2015). *What Every Real Estate Investor Needs to Know About Cash Flow... And 36 Other Key Financial Measures* (3.^a ed.). McGraw-Hill Education.

Geerts, M., Reusens, M., Baesens, B., vanden Broucke, S., & De Weerd, J. (2025). On the performance of LLMs for real estate appraisal. *arXiv preprint*, arXiv:2506.11812. <https://arxiv.org/abs/2506.11812>

Krotov, V., Johnson, L., & Silva, L. (2020). Tutorial: Legality and Ethics of Web Scraping. *Communications of the Association for Information Systems*, 47, Article 22. <https://aisel.aisnet.org/cais/vol47/iss1/22/>

Libertad Inmobiliaria. (2024). *Calculadora de rentabilidad inmobiliaria v3* [Hoja de cálculo]. <https://libertadinmobiliaria.es/calculadora-rentabilidad-inmuebles-alquiler/>

Mitchell, R. (2018). *Web Scraping with Python: Collecting More Data from the Modern Web* (2.^a ed.). O'Reilly Media.

OCU. (2023). Los gastos de la compraventa de vivienda. *OCU Fincas y Casas*. Organización de Consumidores y Usuarios. <https://www.ocu.org/fincas-y-casas/compraventa/compraventa/analisis-gratis/2023/10/gastos-de-compraventa-de-vivienda>

1.17.3 Documentación Técnica

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Meta. (2025). *React documentation*. <https://react.dev/>

Microsoft. (2024). *TypeScript handbook*. <https://www.typescriptlang.org/docs/handbook/>

OpenAI. (2025). *OpenAI API reference*. <https://platform.openai.com/docs/api-reference>

OpenJS Foundation. (2025). *Node.js documentation*. <https://nodejs.org/docs/latest/api/>

SonarSource. (2025). *SonarQube documentation*. <https://docs.sonarqube.org/latest/>

StrongLoop/IBM. (2025). *Express.js documentation*. <https://expressjs.com/>

Tailwind Labs. (2025). *Tailwind CSS documentation*. <https://tailwindcss.com/docs>

Vercel. (2025). *Next.js documentation*. <https://nextjs.org/docs>

1.17.4 Metodología y Buenas Prácticas

Abramov, D. (2015). *Presentational and Container Components* [Blog post]. Medium. https://medium.com/@dan_abramov/smart-and-dumb-components-7ca2f9a7c7d0

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2.^a ed.). Addison-Wesley Professional.

Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship* Prentice Hall.

Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9.^a ed.). McGraw-Hill Education.

1.18 INFORMACIÓN DE CONTACTO Y REPOSITORIO

Proyecto: RealStateAI **Autor:** Alejandro Zabaleta **Universidad:** U-tad **Curso:** 2025-2026

Repositorio: <https://github.com/ZabaHD4K/CalculadoraRentabilidadInmobiliaria> **Frontend en producción:** <https://calculadora-rentabilidad-inmobiliar-six.vercel.app>

Última actualización: 12 de Febrero de 2026 **Estado del proyecto:** En desarrollo activo - ~95% completado (funcionalidad core completa, 49 tests unitarios implementados, frontend refactorizado en 17 componentes, sistema de feedback integrado, validación con usuarios en curso con 100+ testers, pendiente documentación final)

Este documento se actualizará continuamente con los avances del proyecto.